Convolutional Neural Networks for Earth Observation

A Student Reference Textbook

From CNN fundamentals to full geospatial deep-learning pipelines

Backbone project:
Sentinel-2 Multiclass Land-Cover Segmentation

A production-grade course for advanced Python programmers
entering computer vision and remote sensing

Companion to the 12-chapter notebook series and the `eo_cnn` reference package

Edition 1.0 · MIT License (materials)

Contents

Preface

This book is the written reference for the course *Convolutional Neural Networks for Earth Observation*. It is designed for a reader who is already an experienced software engineer and who has trained neural networks before, but who has only seen convolutional networks in theory and has little exposure to satellite imagery. The goal is not to be an encyclopedia of deep learning; it is to take you, chapter by chapter, from the convolution operation to a deployable system that turns a raw Sentinel-2 scene into a georeferenced land-cover map.

Why a single backbone project. Many courses teach CNNs through a scatter of unrelated toy datasets. This one is deliberately built around *one* cohesive arc: land-cover mapping. We begin with patch classification on EuroSAT, where the machinery is simplest, then graduate to dense pixel-level prediction (semantic segmentation) on LoveDA and synthetic Sentinel-2 scenes, and finally assemble a complete inference pipeline. Every concept introduced for classification — the encoder, the loss, transfer learning, augmentation — is reused and extended for segmentation. The continuity is the pedagogy.

How to read this book. Each chapter opens with explicit learning objectives, a difficulty/duration strip, and the datasets it uses. The body develops the theory with the mathematics written out properly, and closes with a *Practical Session* box that mirrors the corresponding Jupyter notebook, followed by exercises and a mini-project. The mathematics is meant to be read with a pencil; the practical boxes are meant to be read with a keyboard. You will get the most out of the book by running the matching notebook (`notebooks/chapter_NN_*`) immediately after each chapter.

Notation. Tensors are written as multidimensional arrays with shape conventions in the PyTorch order (N, C, H, W) — batch, channels, height, width. Scalars are lower-case italic (x), vectors bold lower-case ($\mathbf{x}$), matrices and tensors upper-case (K, I). A convolution layer is denoted $*$ even though, as we will see in Chapter 2, frameworks implement cross-correlation. Spectral bands are referenced by their Sentinel-2 names (B02, B03, …, B12).

Software. All code targets Python 3.12 with PyTorch $\geq$ 2.2, `torchgeo`, `rasterio`, `segmentation-models-pytorch`, and `albumentations`. Every notebook runs on Google Colab's free T4 GPU and falls back to CPU through a `FAST_MODE` flag. Appendix A gives the full environment setup; Appendix B lists papers, tools and datasets.

A note on figures. Several figures in this book are generated programmatically (script `docs/textbook/make_figures.py`) from the same NumPy/Matplotlib stack used in the practicals, so you can reproduce and modify them yourself.

Introduction: Choosing the Backbone Project

Before the first line of code, a curriculum designer must answer one question: *which problem should the whole course be built around?* The choice determines which CNN concepts can be taught organically and which must be bolted on artificially. Three candidate Earth-Observation (EO) projects were evaluated.

The three candidates

Project 1 — EuroSAT land-use classification. Classify 64×64 Sentinel-2 patches into one of ten land-use classes. The dataset (27,000 patches, RGB or 13-band, $\sim$90 MB) downloads in one `torchgeo` call. It is the "MNIST of EO": clean, balanced, and fast on CPU. It is *ideal* for teaching the classification machinery — softmax, cross-entropy, accuracy, training loops — but it teaches no spatial prediction, no geospatial context, and no encoder–decoder design. Difficulty: beginner.

Project 2 — SpaceNet building-footprint extraction. Binary segmentation of buildings from 30 cm very-high-resolution imagery with polygon labels. Strongly motivating and visually satisfying, but the dataset is $\sim$24 GB, requires AWS tooling, uses proprietary Maxar imagery, and demands heavy polygon-to-raster preprocessing before any training. The binary task also limits architectural variety. Difficulty: intermediate to advanced; poor classroom accessibility.

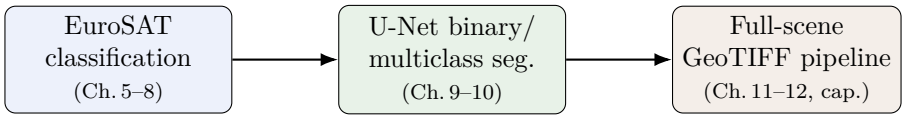
Project 3 — Sentinel-2 multiclass land-cover segmentation. Assign every pixel a land-cover class (urban, agriculture, forest, grassland, water, bare soil, ...) from multispectral 10 m imagery, using LoveDA (auto-downloaded via `torchgeo`, $\sim$3 GB) and synthetic/real Sentinel-2 scenes. It exercises the *entire* CNN curriculum — spatial convolution, pooling, skip connections, transfer learning, augmentation, dense-prediction losses and metrics — and all EO-specific skills: multi-band loading, spectral indices, normalization, tiling, sliding-window inference, and GeoTIFF export. Difficulty: intermediate to advanced, with a beginner on-ramp through EuroSAT.

Comparative evaluation

Criterion	EuroSAT	SpaceNet	S2 land cover
Task type	Classification	Binary segmentation	**Multiclass seg.**
Educational breadth	Narrow	Medium	**Wide**
Dataset access	Excellent (90 MB)	Poor (24 GB, AWS)	**Excellent (3 GB)**
Annotation quality	Excellent	Variable	Good (expert)
Compute needs	Very low	High	Medium (GPU)
Geospatial preproc.	None	Medium	**Full**
Visual output	Low	Medium	**High**
Transfer-learning demo	Good	Good	**Excellent**
Augmentation relevance	Low	Medium	**High**
Industry relevance	Medium	High	**Highest**
Beginner friendliness	**Excellent**	Poor	Good
Full pipeline	No	Partial	**Yes**
Architecture diversity	Low	Medium	**High**
Multi-band / indices	MS variant	No	**Yes**
Overall (1–10)	7	6	**9**

Decision

The backbone of this course is **Sentinel-2 multiclass land-cover segmentation**. It is the only candidate that sustains a coherent twelve-chapter arc without an awkward pivot: EuroSAT classification provides the beginner on-ramp (Chapters 5–8), the same encoder is reused inside a U-Net for segmentation (Chapters 9–10), and the course culminates in full-scene inference and a GeoTIFF map (Chapters 11–12 and the capstone). It mirrors real production systems — ESA WorldCover, Google Dynamic World, Planet NICFI — so the skills transfer directly to industry. SpaceNet's data friction and EuroSAT's pedagogical ceiling rule them out as *sole* backbones, though EuroSAT serves perfectly as the introductory vehicle inside this larger arc.



The shared encoder is the thread: only the decoder head changes as we move from classification to dense prediction.

Part I

Foundations: From Pixels to Convolutions

Chapter 1

Introduction: Earth Observation and Deep Learning

Difficulty: Beginner • **Duration:** ∼4 h (2 h theory + 2 h practical) • **Datasets:** EuroSAT RGB (torchgeo, ∼90 MB)

Learning Objectives

- Understand what satellite imagery is and why it matters for AI.
- Load and visualise multi-band GeoTIFF files with `rasterio`.
- Understand spectral bands, spatial resolution, and coordinate reference systems (CRS).
- Explore the structure of the EuroSAT benchmark dataset.
- Build intuition for why CNNs — not per-pixel classifiers — extract land-use information.

1.1 What is Earth Observation?

Earth Observation (EO) is the systematic collection of information about the Earth's physical, chemical and biological systems using remote sensors, chiefly satellites. Unlike a consumer camera, an EO sensor does not produce a "photograph"; it measures the intensity of electromagnetic radiation reflected or emitted by the surface within a set of narrow wavelength intervals called *spectral bands*. Each band is itself a grey-scale image, so a single acquisition is a stack of co-registered images — a tensor of shape (C, H, W) where C is the number of bands. This structural identity with the input a CNN expects is the reason deep learning transferred so naturally to remote sensing.

Four properties characterise any EO sensor and constrain the models we build on top of it:

Spatial resolution
the ground area covered by one pixel (the ground-sampling distance, GSD). Sentinel-2 is 10–60 m; WorldView-3 reaches 0.3 m.

Spectral resolution
the number and width of the bands. More bands reveal more about surface chemistry (e.g. chlorophyll, moisture).

Temporal resolution
the revisit period. Sentinel-2 revisits every ∼5 days, enabling change-detection and time-series tasks.

Radiometric resolution
the bit depth of the measurement (Sentinel-2 stores 12-bit reflectance scaled to integers).

Satellite	Operator	Resolution	Revisit	Bands
Sentinel-2	ESA	10–60 m	5 days	13
Landsat 8/9	USGS/NASA	30 m	16 days	11
WorldView-3	Maxar	0.3 m	1 day	29
Planet NICFI	Planet	4.77 m	daily	4

Resolution drives architecture. At 10 m/px a single Sentinel-2 pixel covers a tennis court, so individual buildings are sub-pixel; at 0.3 m/px cars and tree crowns are resolved. The size of the objects you care about, measured in pixels, must fit inside the network's receptive field (Chapter 2).

1.2 Spectral bands: what satellites measure

The single most important EO concept for a newcomer is that *each band is an image*, and different materials have characteristic *spectral signatures* — reflectance as a function of wavelength. Figure 1.1 shows idealised signatures for four land-cover types across the Sentinel-2 bands.

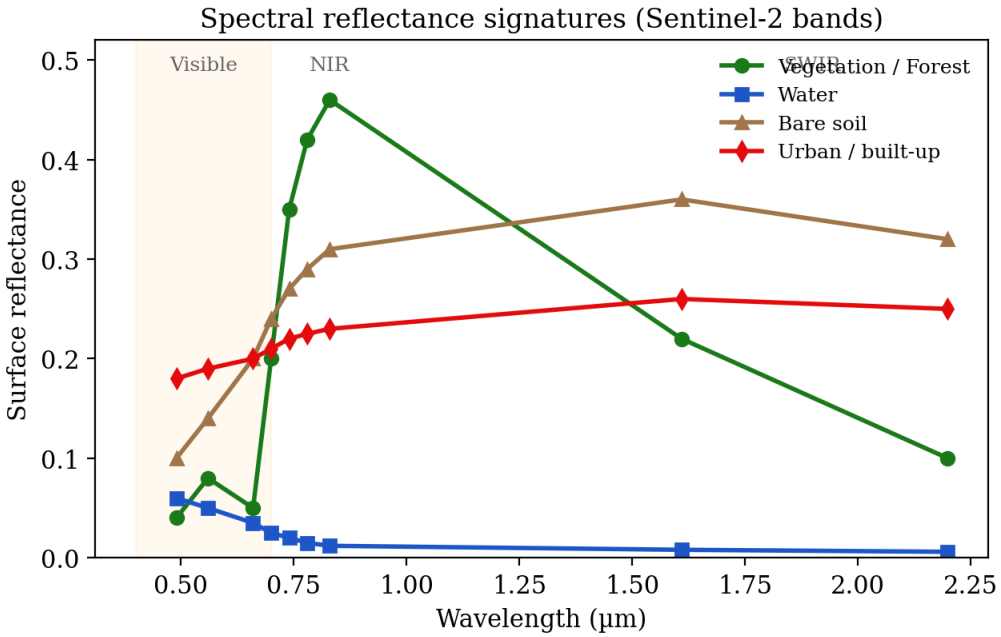


Figure 1.1: Spectral reflectance signatures. Healthy vegetation reflects strongly in the near-infrared (NIR, B08) and absorbs in the red (B04); water absorbs almost everything beyond the visible; soil rises gently toward the SWIR. These contrasts are exactly what spectral indices and CNNs exploit.

The vegetation "red edge" — the steep rise between B04 (red) and B08 (NIR) — underlies the most famous remote-sensing index, the Normalised Difference Vegetation Index:

$$\mathrm{NDVI} = \frac{\rho_{\mathrm{NIR}} - \rho_{\mathrm{Red}}}{\rho_{\mathrm{NIR}} + \rho_{\mathrm{Red}}} = \frac{\mathrm{B08} - \mathrm{B04}}{\mathrm{B08} + \mathrm{B04}} \in [-1, 1]. \tag{1.1}$$

Dense vegetation gives NDVI $\gtrsim 0.6$; water gives negative values. We will compute indices like this in Chapter 4 and feed them as extra channels to networks.

Common pitfall

Reflectance values are not 8-bit "pixels". Sentinel-2 Level-2A products store bottom-of-atmosphere reflectance as integers (e.g. 0–10000). Forgetting to scale and normalise these before feeding a network is the single most frequent beginner error (Chapter 4).

1.3 Raster and vector data

GIS data comes in two forms. *Raster* data is a grid of pixels with a geotransform mapping pixel (row, col) to a world coordinate, plus a CRS (coordinate reference system, e.g. EPSG:32633 for

UTM zone 33N). Satellite imagery and our prediction maps are rasters, read and written with `rasterio`. *Vector* data is geometry — points, lines, polygons — stored in shapefiles or GeoJSON and manipulated with `geopandas`; building footprints and OpenStreetMap labels are vectors. Training a segmentation model usually means *rasterising* vector labels onto the same grid as the imagery.

1.4 The EuroSAT dataset

EuroSAT (Helber et al., 2019) is the benchmark we use to learn the classification machinery. It contains 27,000 Sentinel-2 patches of 64×64 pixels, each labelled with one of ten land-use classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake. Two variants exist: RGB (3 bands) and multispectral (13 bands). It is clean, balanced ($\sim$2000–3000 samples/class) and downloads in one line via `torchgeo`, which makes it perfect for fast iteration — but note that a patch-level label teaches *classification only*; it cannot teach the spatial prediction we build toward later.

1.5 Why CNNs, not per-pixel classifiers?

Classical remote sensing classified each pixel independently from its spectral vector using SVMs or random forests. This ignores *spatial context*. Consider the visual texture of land cover: forests have a rough, self-similar canopy; urban areas show grid-like street patterns and high local heterogeneity; water is smooth and homogeneous; agriculture shows regular field boundaries and row striping. None of these are visible to a per-pixel method, because they live in the *neighbourhood* of a pixel, not in the pixel itself. A convolutional network learns spatial filters that respond to exactly such patterns, and stacks them into a hierarchy — the subject of the next three chapters.

Key Concepts

- Each spectral band is an image; an acquisition is a (C, H, W) tensor.
- Spectral signatures distinguish materials; indices such as NDVI (Eq. 1.1) summarise them.
- Reflectance must be scaled and normalised before training.
- Raster (pixels + CRS) vs vector (geometry); labels are often rasterised.
- CNNs learn *spatial* patterns that per-pixel classifiers miss.

Practical Session

Notebook: `notebooks/chapter_01_intro_eo_dl/01_practical_eo_data_exploration.ipynb`.

1. Run the Colab/environment setup cell; confirm `DEVICE` and `FAST_MODE` are detected.
2. Download EuroSAT RGB via `torchgeo` and visualise one sample per class in a 2×5 grid.
3. Inspect the class distribution as a bar chart; confirm the dataset is near-balanced.
4. Read a multi-band GeoTIFF with `rasterio`; print its shape, dtype, CRS and geotransform; render a true-colour composite from B04/B03/B02.
5. Compute and display NDVI for a vegetated and a water sample.

A representative skeleton for reading a scene and computing an index:

```
import rasterio, numpy as np
with rasterio.open(path) as src:
    img = src.read().astype("float32")   # (C, H, W)
    print(img.shape, src.crs, src.transform)
red, nir = img[3], img[7]                # B04, B08 (0-indexed)
ndvi = (nir - red) / (nir + red + 1e-6)
```

Exercises & Mini-Project

1.1 Channel statistics. Over the EuroSAT test split, compute the per-channel mean and standard deviation. Tabulate them — these become your normalization constants in Chapter 5.

1.2 Spectral profiles. For five random samples from each class, plot the mean reflectance vs band number. Verify water has near-zero NIR/SWIR and forest has high NIR.

1.3 NDVI thresholding. Using EuroSAT-MS, compute NDVI for three Forest and three SeaLake samples; find a threshold separating them.

Mini-project. Write `describe_sample(idx)` that prints the class name, RGB and NIR means, and shows the RGB image beside an NDVI heatmap; run it on five samples.

Chapter 2

Convolution Fundamentals

Difficulty: Beginner • **Duration:** ∼5 h (3 h theory + 2 h practical) • **Datasets:** EuroSAT; synthetic images

Learning Objectives

- Define discrete 2D convolution and distinguish it from cross-correlation.
- Implement convolution from scratch in NumPy and reproduce `nn.Conv2d`.
- Derive the output-size formula for arbitrary kernel, stride and padding.
- Understand kernels, feature maps, weight sharing and the receptive field.
- Read and reason about every argument of `torch.nn.Conv2d`.

2.1 The convolution operation

A convolution slides a small array of learnable weights — the *kernel* or *filter* K — across an input image I and, at each position, computes the sum of element-wise products. The result is a *feature map*. For a single-channel input of size $H \times W$ and a kernel of size $k \times k$, the (valid) output at position (i, j) is

$$S[i,j] \;=\; (I * K)[i,j] \;=\; \sum_{m=0}^{k-1}\sum_{n=0}^{k-1} K[m,n]\, I[i+m,\, j+n]. \tag{2.1}$$

Figure 2.1 illustrates one such dot-product for a vertical-edge kernel.

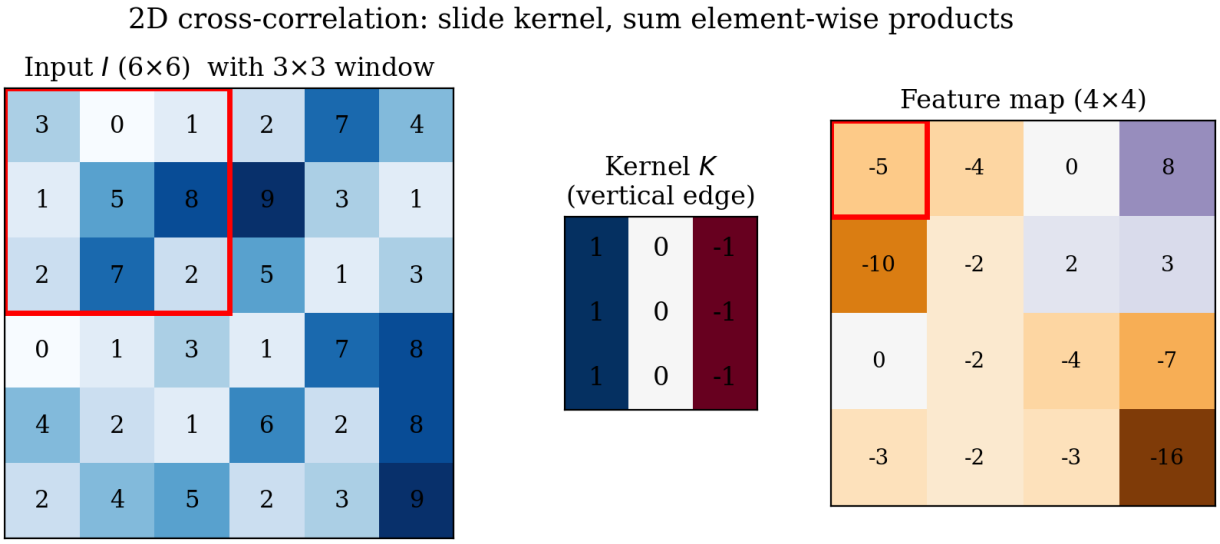


Figure 2.1: Discrete 2D cross-correlation. The 3×3 kernel (centre) is placed at every valid position of the input (left); each placement produces one output value (right). The same kernel is reused everywhere — *weight sharing*.

Convolution vs. cross-correlation. The mathematical convolution flips the kernel before sliding: $(I * K)[i, j] = \sum_{m,n} K[m, n]\, I[i - m, j - n]$. Deep-learning frameworks omit the flip and implement Eq. (2.1) — technically *cross-correlation* — but call it "convolution". Because the kernel weights are learned, the flip is irrelevant: the network simply learns the un-flipped kernel. We follow the framework convention throughout.

Multi-channel convolution. Real inputs have C_{in} channels (3 for RGB, 13 for Sentinel-2). A single filter then has shape (C_{in}, k, k) and sums over channels; a layer learns C_{out} such filters, producing C_{out} feature maps:

$$S_o[i, j] \;=\; b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K_o[c, m, n]\, I[c,\, i + m,\, j + n], \qquad o = 0, \ldots, C_{\text{out}} - 1. \tag{2.2}$$

The weight tensor of a layer therefore has shape $(C_{\text{out}}, C_{\text{in}}, k, k)$ and the bias has shape (C_{out}).

Why weight sharing matters. A fully connected layer mapping a $64 \times 64 \times 3$ image to even 64 hidden units needs $64 \times 64 \times 3 \times 64 \approx 786{,}000$ weights and is not translation-equivariant. A convolution with 64 filters of size $3 \times 3 \times 3$ needs $64 \times 27 = 1{,}728$ weights, applies the *same* detector everywhere, and is translation-equivariant by construction. This parameter economy and built-in spatial prior are why CNNs dominate imagery.

2.2 Classic kernels: what filters detect

Before learned filters, image processing used hand-designed kernels. They build intuition for what a CNN's early layers discover on their own. Figure 2.2 applies several to a synthetic satellite-like scene.

The horizontal Sobel kernel is $K = \left(\begin{smallmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{smallmatrix}\right)$; it approximates $\partial I / \partial x$ and fires on vertical edges. Early CNN layers learn Gabor-like oriented edge and colour-opponent detectors; deeper layers combine these into textures, object parts and, eventually, semantic concepts — a learned hierarchy of the hand-built filters above.

2.3 Padding and stride

Padding. Valid convolution shrinks the output: each spatial dimension loses $k - 1$ pixels. To keep the spatial size constant ("same" padding) we pad the border with $p = \lfloor k/2 \rfloor$ zeros on each side.

Stride. The stride s is the step between kernel placements. A stride of 2 halves the output resolution, providing a learnable alternative to pooling for downsampling.

Output-size formula. For input size n, kernel k, padding p, stride s:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \tag{2.3}$$

For $n = 64$, $k = 3$, $s = 1$: $p = 0$ gives 62 (shrinks); $p = 1$ gives 64 (same). With $s = 2$, $k = 3$, $p = 1$: $n_{\text{out}} = 32$. Memorising Eq. (2.3) prevents the most common shape-mismatch bugs.

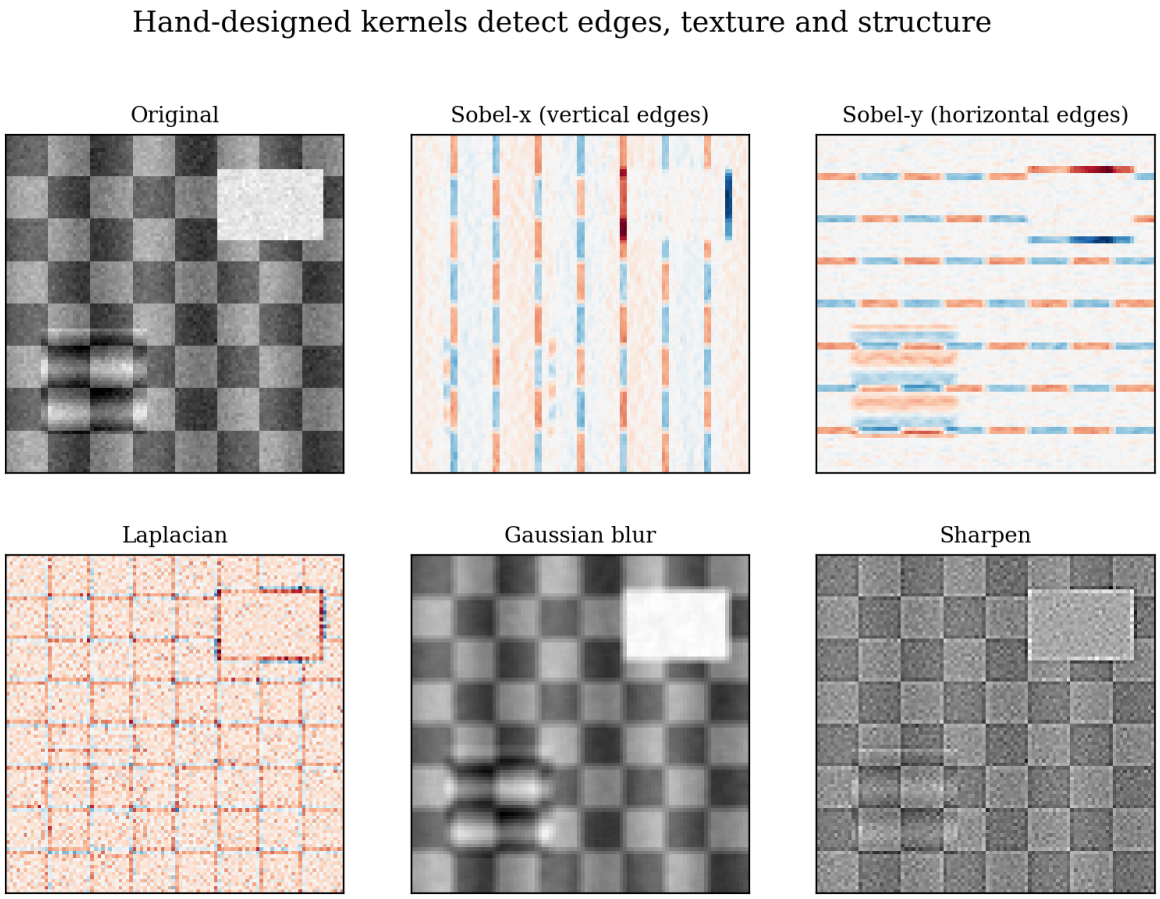


Figure 2.2: Hand-designed kernels. Sobel filters respond to oriented edges; the Laplacian to second-order intensity changes; Gaussian blurs; sharpening amplifies high frequencies. Trained CNN first layers typically rediscover edge/blob/colour-opponent filters resembling these.

2.4 The receptive field

The *receptive field* (RF) of a unit is the region of the input image that can influence its activation. For N stacked $k \times k$ convolutions with stride 1 and no pooling,

$$\mathrm{RF}_N = 1 + N\,(k-1). \tag{2.4}$$

For 3×3 kernels: one layer sees 3, two layers 5, five layers 9, ten layers 19. Stacking small kernels is cheaper than one large kernel for the same RF (two 3×3 layers, 18 weights/channel, match one 5×5, 25 weights) and inserts an extra non-linearity — the design principle behind VGG (Chapter 6). Pooling or strided convolution multiplies the RF growth rate (Figure 2.3).

This is why depth lets a network detect larger objects: a unit in a deep layer integrates information over a wide spatial extent. For EO this maps directly to ground distance: matching the RF to the scale of your target features (a road vs a city) is a real design decision.

2.5 `nn.Conv2d` in PyTorch

The framework layer realises Eq. (2.2):

```
import torch.nn as nn
conv = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3,
                 stride=1, padding=1, bias=True)
# weight shape: (out_channels, in_channels, kH, kW) = (32, 3, 3, 3)
```

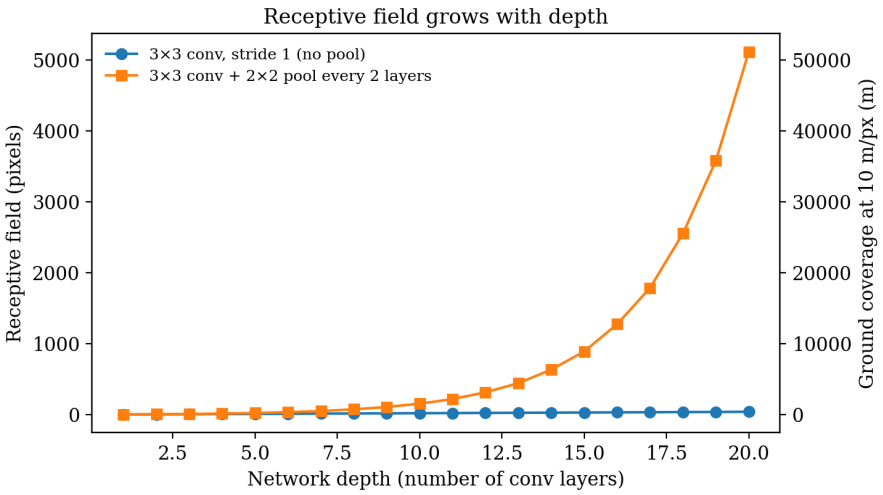


Figure 2.3: Receptive field vs depth. Downsampling layers (pooling or stride 2) make the RF grow geometrically, letting deep layers "see" large ground areas. At 10 m/px an RF of ∼100 px corresponds to ∼1 km — enough to capture field boundaries and urban blocks.

```
# params: out*in*kH*kW + out = 32*3*3*3 + 32 = 896
```

Key arguments: `in_channels` (input feature maps), `out_channels` (number of filters learned = output maps), `kernel_size`, `stride`, `padding`, and `bias` (set `False` when a BatchNorm layer follows, since BN has its own shift). The parameter count of a layer is $C_{\text{out}}(C_{\text{in}}\,k^2) + C_{\text{out}}$; its cost in multiply–accumulate operations for an $H \times W$ output is $C_{\text{out}}C_{\text{in}}k^2HW$.

Key Concepts

- Convolution = slide kernel, sum element-wise products (Eq. 2.1); frameworks skip the flip.
- A layer learns C_{out} filters of shape (C_{in}, k, k); weight sharing gives translation equivariance and few parameters.
- Output size: $\lfloor (n + 2p - k)/s \rfloor + 1$. Same padding: $p = \lfloor k/2 \rfloor$.
- Receptive field grows with depth (Eq. 2.4); pooling/stride accelerate it.
- Stacked 3×3 convolutions beat one large kernel: fewer params, more non-linearity.

Practical Session

Notebook: `chapter_02_convolution_fundamentals/02_practical_convolution_from_scratch.ipynb`.

1. Implement `convolve2d_numpy(img, kernel)` for valid convolution and verify it against `scipy.signal.correlate2d`.
2. Apply Sobel-x, Sobel-y, Laplacian, Gaussian-blur and sharpen kernels to an EuroSAT sample; display the feature maps.
3. Verify Eq. (2.3) empirically for several (k, p, s) triples.
4. Recreate the NumPy convolution with `nn.Conv2d` by manually setting its `weight`; confirm the outputs match.
5. Trace the receptive field of a 5-layer stack using Eq. (2.4).

```python
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view
def convolve2d_numpy(img, k):
    win = sliding_window_view(img, k.shape)      # (H-k+1, W-k+1, k, k)
    return np.einsum("ijmn,mn->ij", win, k)
```

Exercises & Mini-Project

2.1 Strided convolution. Extend `convolve2d_numpy` to support `stride>1`; verify the output size against Eq. (2.3).

2.2 Parameter & MAC counting. For each layer of the Chapter 3 `SimpleCNN`, compute learnable parameters and multiply–accumulates for a 64×64 input (MACs $= C_{\text{out}} C_{\text{in}} k^2 H_{\text{out}} W_{\text{out}}$).

2.3 Kernel visualisation. Train `SimpleCNN` for two epochs (Chapter 5 code) and display the 32 first-layer filters; compare with random initialisation. Do Gabor-like patterns emerge?

Mini-project. Hand-design three 3×3 kernels detecting (i) diagonal edges, (ii) a regular grid texture, (iii) a colour-difference response across channels; apply them to five EuroSAT samples and interpret the maps.

Chapter 3

CNN Building Blocks

Difficulty: Beginner • **Duration:** ∼4 h (2 h theory + 2 h practical) • **Datasets:** EuroSAT; synthetic data

Learning Objectives

- Understand pooling (max/average) and global pooling, and their effect on resolution.
- Compare activation functions and know when to use each.
- Explain batch normalisation and why it stabilises and accelerates training.
- Use dropout to regularise, and recognise the signature of overfitting.
- Assemble these into a complete, well-reasoned CNN classifier.

A convolution layer alone is a linear operator. A working CNN interleaves it with non-linearities, downsampling, normalisation and regularisation. This chapter covers those building blocks and assembles them into the `SimpleCNN` we train in Chapter 5.

3.1 Pooling

Pooling downsamples a feature map, reducing spatial resolution (and therefore computation) while enlarging the effective receptive field. *Max pooling* takes the maximum over each $p \times p$ window; *average pooling* takes the mean (Figure 3.1):

$$\mathrm{MaxPool}(X)[i,j] = \max_{0 \le m,n < p} X[si+m,\, sj+n], \quad \mathrm{AvgPool}(X)[i,j] = \frac{1}{p^2} \sum_{m,n} X[si+m,\, sj+n]. \tag{3.1}$$

Max pooling preserves the strongest activation (good for detecting whether a feature is *present*); average pooling smooths. Most modern networks use strided convolutions or max pooling in the body and a *global average pool* (GAP) at the end, collapsing each $C \times H \times W$ feature map to a C-vector by averaging over space. GAP removes the need for large fully connected layers, drastically cutting parameters and overfitting, and underlies class-activation maps (Chapter 12).

3.2 Activation functions

Without a non-linearity, a stack of convolutions collapses to a single linear map. The activation ϕ applied element-wise after each layer gives the network its expressive power (Figure 3.2). The default is the rectified linear unit:

$$\mathrm{ReLU}(z) = \max(0, z). \tag{3.2}$$

ReLU is cheap, non-saturating for $z > 0$, and induces sparsity, but "dead" units can occur when a neuron is pushed permanently negative. *LeakyReLU*, $\phi(z) = \max(\alpha z, z)$ with small α, keeps a gradient for $z < 0$. *GELU*, $\phi(z) = z\,\Phi(z)$ (with Φ the Gaussian CDF), is a smooth variant favoured in transformers. The saturating *sigmoid* $\sigma(z) = 1/(1+e^{-z})$ and *tanh* are now used mainly in output layers (binary probability) or gates, because their vanishing gradients slow deep training.

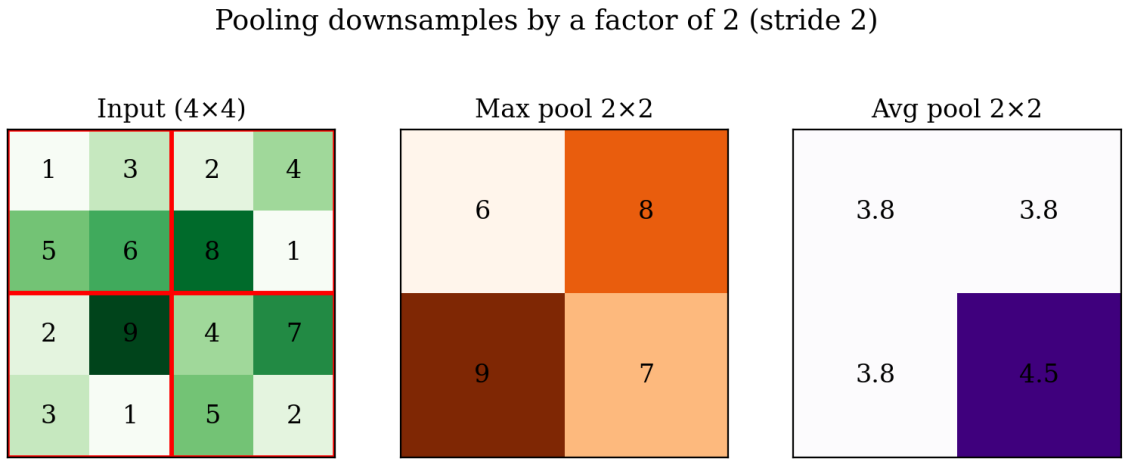


Figure 3.1: Max vs average pooling with a 2×2 window and stride 2. Both halve each spatial dimension; max pooling keeps the dominant response, average pooling the local mean.

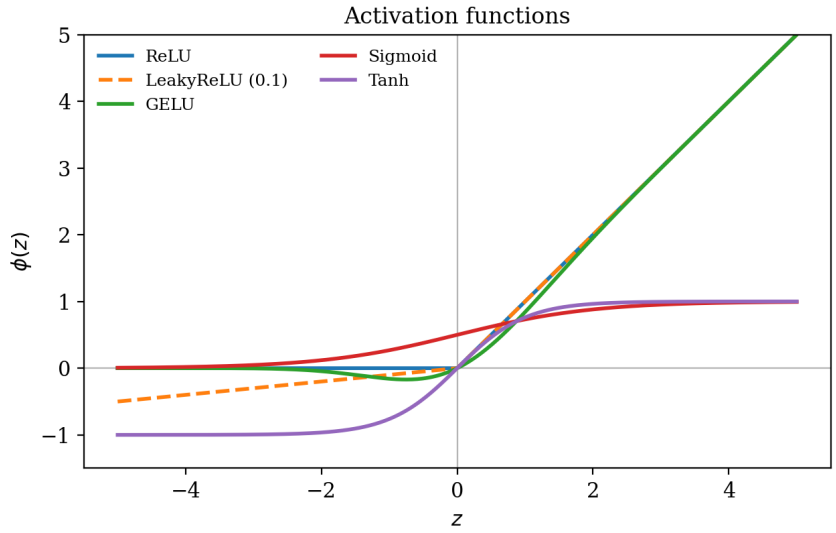


Figure 3.2: Common activation functions. ReLU and its smooth relatives (LeakyReLU, GELU) dominate hidden layers; sigmoid/tanh saturate and are reserved for outputs and gates.

3.3 Batch normalisation

Batch normalisation (Ioffe & Szegedy, 2015) normalises each channel's pre-activations across the mini-batch, then rescales with learnable parameters γ, β. For a channel with batch values $\{x_i\}$:

$$\mu_B = \frac{1}{m}\sum_i x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_i (x_i - \mu_B)^2, \tag{3.3}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma\,\hat{x}_i + \beta. \tag{3.4}$$

BatchNorm reduces *internal covariate shift*, smooths the loss landscape, permits higher learning rates, and acts as a mild regulariser. At inference it uses running estimates of μ, σ accumulated during training. Because BN already provides a per-channel shift β, the preceding convolution sets `bias=False`. Place it as `Conv` $\rightarrow$ `BN` $\rightarrow$ `ReLU`.

Small-batch caveat

BatchNorm statistics are unreliable with very small batches (common for high-resolution segmentation on limited VRAM). In that regime prefer GroupNorm or InstanceNorm, which normalise without a batch dependency.

3.4 Dropout and overfitting

Overfitting occurs when a model memorises training data and fails to generalise; its signature is a training loss that keeps falling while validation loss rises. *Dropout* (Srivastava et al., 2014) combats this by randomly zeroing a fraction p of activations during training, scaling the survivors by $1/(1-p)$, which prevents co-adaptation and approximates an ensemble. It is disabled at inference. In convolutional bodies dropout is used sparingly (spatial structure makes element-wise dropout weak); it is most effective in the dense head. The stronger regularisers for CNNs are data augmentation (Chapter 8), weight decay, and BatchNorm itself.

3.5 Assembling a CNN

A canonical small classifier stacks *blocks* of `Conv`→`BN`→`ReLU`→`Pool`, doubling channels as resolution halves, then a GAP and a linear classifier:

```
import torch.nn as nn
class ConvBlock(nn.Module):
    def __init__(self, cin, cout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(cin, cout, 3, padding=1, bias=False),
            nn.BatchNorm2d(cout), nn.ReLU(inplace=True),
            nn.MaxPool2d(2))            # halves H, W
    def forward(self, x): return self.net(x)

class SimpleCNN(nn.Module):
    def __init__(self, in_ch=3, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(in_ch, 32),       # 64 -> 32
            ConvBlock(32, 64),          # 32 -> 16
            ConvBlock(64, 128))         # 16 -> 8
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Dropout(0.3), nn.Linear(128, n_classes))
    def forward(self, x): return self.head(self.features(x))
```

The channel-doubling/resolution-halving rule keeps the per-layer compute roughly constant while building an increasingly abstract, lower-resolution representation — a pattern you will see again in every ResNet and U-Net encoder.

Key Concepts

- Pooling/strides downsample and grow the RF; GAP replaces heavy FC heads.
- ReLU is the default hidden activation; sigmoid/tanh for outputs/gates.
- BatchNorm (Eq. 3.4) stabilises and speeds training; order is Conv→BN→ReLU, bias off.
- Overfitting = train loss ↓ while val loss ↑; fight it with augmentation, weight decay, dropout.
- Standard pattern: blocks of Conv-BN-ReLU-Pool, doubling channels, then GAP + linear.

Practical Session

Notebook: `chapter_03_cnn_building_blocks/03_practical_cnn_components.ipynb`.

1. Visually compare max vs average pooling on a sample feature map.
2. Plot ReLU, LeakyReLU, GELU, sigmoid and tanh and their gradients.
3. Train a small net *with* and *without* BatchNorm; compare loss curves and the largest stable learning rate.
4. Force overfitting on a tiny subset, then add dropout and observe the validation-loss gap shrink.
5. Build `SimpleCNN`; print a layer-by-layer summary of shapes and parameter counts.

Exercises & Mini-Project

3.1 Architecture ablation. Vary depth (2 vs 3 vs 4 blocks) and width (16/32/64 base channels); tabulate parameters, train time and accuracy.

3.2 Normalisation swap. Replace BatchNorm with GroupNorm; retrain at batch size 4 and explain the difference.

3.3 Activation study. Swap ReLU for LeakyReLU and GELU; measure convergence speed and final accuracy.

Mini-project. Implement a configurable `make_cnn(depth, width, norm, act, dropout)` factory and run a small grid search on EuroSAT, reporting a parameters-vs-accuracy scatter plot.

Chapter 4

Remote Sensing Data and Geospatial Preprocessing

Difficulty: Beginner → Intermediate • **Duration:** ∼5 h (2.5 h + 2.5 h) • **Datasets:** Sentinel-2 sample tiles; synthetic scenes

Learning Objectives

- Read and write multi-band GeoTIFFs with `rasterio` and `rioxarray`.
- Understand CRS, geotransforms, and band resolution mismatches.
- Compute spectral indices (NDVI, NDWI, NDBI, EVI) as engineered channels.
- Choose and apply a normalization strategy for reflectance data.
- Tile a large scene into model-ready patches and reconstruct it.

This is the first genuinely Earth-Observation chapter: turning a raw multispectral scene into tensors a CNN can train on. The steps — loading, index computation, normalization, tiling — form the front half of every pipeline in the rest of the book.

4.1 Reading multi-band rasters

A GeoTIFF couples a pixel grid with georeferencing: a CRS and an affine *geotransform* mapping array indices to world coordinates,

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a & b \\ d & e \end{pmatrix} \begin{pmatrix} \text{col} \\ \text{row} \end{pmatrix} + \begin{pmatrix} c \\ f \end{pmatrix}, \tag{4.1}$$

where (c, f) is the top-left corner and a, e are the pixel sizes (with $e < 0$). `rasterio` exposes this as `src.transform` and the projection as `src.crs`.

```
import rasterio
with rasterio.open("scene.tif") as src:
    arr = src.read()          # (bands, H, W), native dtype (uint16 reflectance)
    profile = src.profile      # crs, transform, dtype, nodata, ...
```

`rioxarray` wraps the same data as a labelled `xarray.DataArray` with named dimensions and coordinates, which is convenient for band maths and resampling. A practical complication is that Sentinel-2 bands have *mixed resolutions* (10/20/60 m); to stack them into one tensor you resample the 20/60 m bands to the 10 m grid (bilinear for continuous reflectance).

4.2 Spectral indices as engineered features

Normalised-difference indices are cheap, physically motivated channels that often boost segmentation accuracy on the classes they target:

$$\mathrm{NDVI} = \frac{\mathrm{B08} - \mathrm{B04}}{\mathrm{B08} + \mathrm{B04}} \qquad \text{(vegetation)}, \tag{4.2}$$

$$\mathrm{NDWI} = \frac{\mathrm{B03} - \mathrm{B08}}{\mathrm{B03} + \mathrm{B08}} \qquad \text{(open water)}, \tag{4.3}$$

$$\mathrm{NDBI} = \frac{\mathrm{B11} - \mathrm{B08}}{\mathrm{B11} + \mathrm{B08}} \qquad \text{(built-up)}, \tag{4.4}$$

$$\mathrm{EVI} = 2.5\,\frac{\mathrm{B08} - \mathrm{B04}}{\mathrm{B08} + 6\,\mathrm{B04} - 7.5\,\mathrm{B02} + 1} \qquad \text{(canopy, haze-robust)}. \tag{4.5}$$

Figure 4.1 shows a synthetic scene and its NDVI: forest and crops light up, water goes negative. Appending such indices as extra channels gives the network features it might otherwise have to learn from scratch.

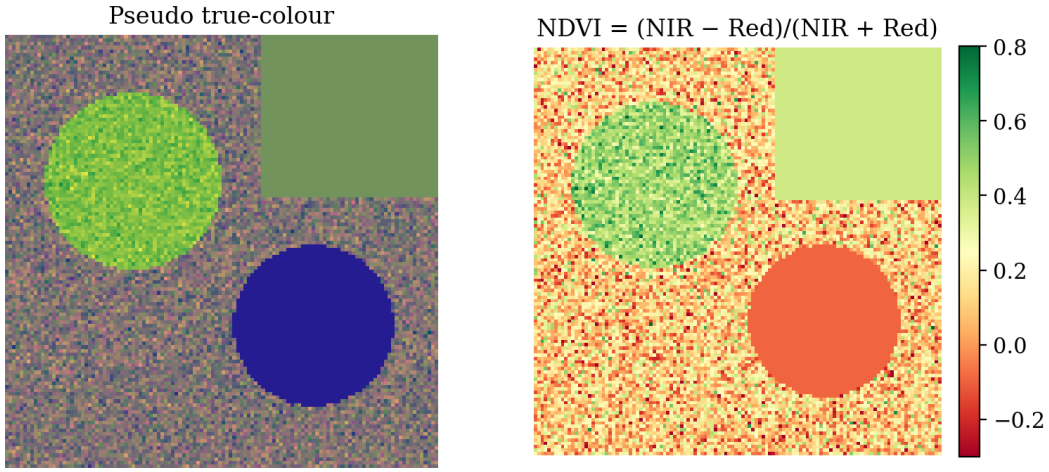


Figure 4.1: A pseudo true-colour scene (left) and its NDVI (right). High NDVI (green) marks vegetation; water is strongly negative (red). Indices encode domain knowledge as ready-made input channels.

4.3 Normalization strategies

Networks train best on inputs with comparable, $O(1)$ scales. Raw Sentinel-2 digital numbers span 0–10000 and differ per band, so normalization is mandatory (Figure 4.2). Three common strategies:

Min–max
$x' = (x - x_{\min})/(x_{\max} - x_{\min}) \in [0, 1]$. Simple but sensitive to outliers (a single bright cloud rescales everything).

Percentile clip
map the 2nd–98th percentile to $[0, 1]$ and clip. Robust to outliers; the standard for visualisation and a strong default for training.

Z-score
$x' = (x - \mu_c)/\sigma_c$ with *per-band* dataset statistics μ_c, σ_c. Pairs naturally with ImageNet-pretrained encoders (which expect standardised inputs).

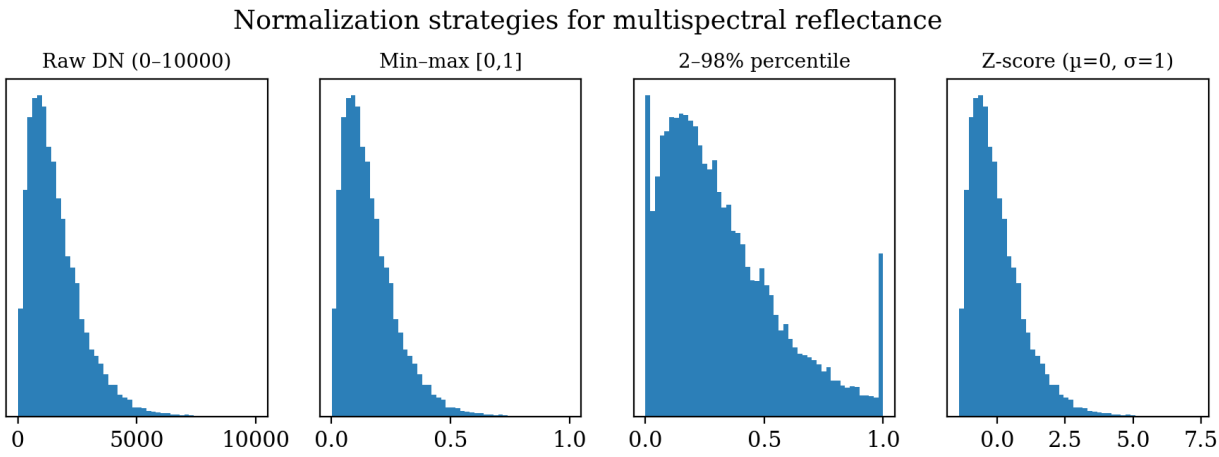


Figure 4.2: Effect of normalization on a reflectance histogram. Percentile clipping tames the long bright tail; z-scoring centres each band at zero with unit variance — the form pretrained encoders expect.

Train/serve consistency

Compute normalization statistics on the *training* set only and reuse them unchanged for validation, test and inference. Recomputing per-image or per-split statistics leaks information and breaks reproducibility in the deployed pipeline.

4.4 Tiling a scene

Sentinel-2 tiles are ∼10980 × 10980 pixels — far too large for a GPU. We cut them into fixed patches (e.g. 512 × 512) with an overlap so that the later sliding-window inference (Chapter 11) can blend predictions and avoid seams. Given tile size T and stride $s < T$ (overlap $T - s$), tile origins are $\{(is, js)\}$ clipped to the scene; edge tiles are padded. Reconstruction reverses the process, accumulating predictions and dividing by a weight map. A reusable preprocessing class typically chains: load → resample → compute indices → normalize → tile, returning (C, T, T) tensors plus each tile's geotransform for georeferenced export later.

Key Concepts

- GeoTIFF = pixels + CRS + affine geotransform; read with `rasterio`/`rioxarray`.
- Resample mixed-resolution bands to a common grid before stacking.
- Spectral indices (NDVI/NDWI/NDBI/EVI) are cheap, informative extra channels.
- Normalize (percentile clip or per-band z-score) using *training* statistics only.
- Tile large scenes with overlap; keep each tile's transform for export.

Practical Session

Notebook: `chapter_04_geospatial_preprocessing/04_practical_sentinel2_preprocessing.ipynb`.

1. Generate (or download) a synthetic multi-band Sentinel-2 GeoTIFF; inspect shape, CRS, transform and per-band ranges with `rasterio`.
2. Render true-colour and false-colour (NIR-Red-Green) composites.
3. Load with `rioxarray`; compute NDVI and NDWI as new DataArrays.
4. Compare min–max, percentile and z-score normalization via histograms.
5. Implement `tile_scene` / `reconstruct_from_tiles` and verify a round-trip reconstructs the input exactly.

```python
def percentile_norm(band, lo=2, hi=98):
    p_lo, p_hi = np.percentile(band, [lo, hi])
    return np.clip((band - p_lo) / (p_hi - p_lo + 1e-6), 0, 1)
```

Exercises & Mini-Project

4.1 Real data. Download a real Sentinel-2 L2A tile (e.g. via the Copernicus browser or `sentinelsat`); load four 10 m bands and resample two 20 m bands to 10 m.

4.2 Index zoo. Implement NDVI, NDWI, NDBI and EVI as functions; display each over a real scene and interpret the highlighted land cover.

4.3 Overlap study. Tile a scene at 0%, 25% and 50% overlap; count tiles and discuss the inference cost/quality trade-off.

Mini-project. Build a `Sentinel2Tile` class that loads bands, appends two indices, percentile-normalizes, and yields georeferenced 512×512 tiles ready for a `DataLoader`.

Part II

Classification: The First Complete Pipeline

Chapter 5

The First Complete Classification Pipeline

Difficulty: Intermediate • **Duration:** ∼5 h (2 h + 3 h) • **Datasets:** EuroSAT (torchgeo)

Learning Objectives

- Derive softmax and the cross-entropy loss for multi-class classification.
- Build train/validation/test `DataLoader`s from a `torchgeo` dataset.
- Write a complete training loop with validation, checkpointing and metrics.
- Diagnose training from loss/accuracy curves.
- Evaluate on a held-out test set and interpret the result.

We now assemble the building blocks of Chapter 3 into an end-to-end pipeline that trains `SimpleCNN` to classify EuroSAT patches. This is the template every later pipeline reuses — only the data, model and loss change.

5.1 Softmax and cross-entropy

A classifier outputs a vector of *logits* $\mathbf{z} \in \mathbb{R}^K$ (one per class). The softmax turns logits into a probability distribution:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_k p_k = 1. \tag{5.1}$$

For a sample with true class y, the (categorical) cross-entropy loss is the negative log-likelihood of the correct class:

$$\mathcal{L}_{\mathrm{CE}} = -\log p_y = -z_y + \log \sum_j e^{z_j}. \tag{5.2}$$

Its gradient with respect to the logits is strikingly simple,

$$\frac{\partial \mathcal{L}_{\mathrm{CE}}}{\partial z_k} = p_k - \mathbb{1}[k = y], \tag{5.3}$$

i.e. "predicted minus target". The loss explodes when the model is confidently wrong ($p_y \to 0$) and vanishes when confidently right (Figure 5.1). In PyTorch, `nn.CrossEntropyLoss` fuses log-softmax and NLL for numerical stability and expects *raw logits* — never apply softmax before it.

5.2 Data loading with torchgeo

`torchgeo` downloads EuroSAT and yields samples as dictionaries (`{"image":...,"label":...}`), so we provide a small `collate_fn` and wrap splits in `DataLoader`s. We normalize with the per-channel statistics computed in Chapter 1 and shuffle only the training split.

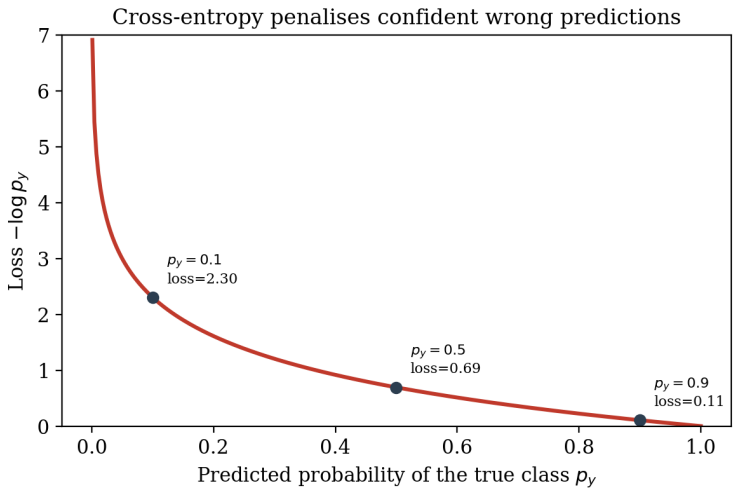


Figure 5.1: Cross-entropy as a function of the predicted probability of the true class. The steep penalty near $p_y = 0$ is what drives the network to be both correct and calibrated.

```python
from torch.utils.data import DataLoader, random_split
def collate(batch):
    imgs = torch.stack([b["image"].float()/3000. for b in batch])
    lbls = torch.tensor([int(b["label"]) for b in batch])
    return imgs, lbls
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True,
                      num_workers=2, collate_fn=collate)
```

A correct split is non-negotiable: the test set is touched *once*, at the very end. Use a fixed seed so splits are reproducible.

5.3 The training loop

One epoch iterates the training loader, and for each batch performs the five canonical steps: forward, loss, backward, step, zero-grad. After each epoch we evaluate on the validation set (no gradients) and checkpoint the best model.

```python
import torch
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
crit = torch.nn.CrossEntropyLoss()
for epoch in range(EPOCHS):
    model.train()
    for x, y in train_dl:
        x, y = x.to(DEVICE), y.to(DEVICE)
        opt.zero_grad()
        loss = crit(model(x), y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        opt.step()
    val_acc = evaluate(model, val_dl)        # no_grad, argmax == y
    if val_acc > best: torch.save(model.state_dict(), "best.pth")
```

Several details matter in practice: `model.train()`/`model.eval()` toggle BatchNorm and dropout; `torch.no_grad()` during validation saves memory; gradient clipping guards against rare exploding updates; and AdamW's decoupled weight decay regularises. A learning-rate scheduler (cosine or step) usually adds a point or two of accuracy.

5.4 Reading the curves

Plot training and validation loss and accuracy every epoch (Figure 5.2). A healthy run shows both losses descending and then plateauing close together. A growing gap — validation loss rising while training loss falls — signals overfitting and calls for more augmentation (Chapter 8), weight decay, or early stopping. A validation loss that never descends usually means a too-small/too-large learning rate or a data bug (e.g. unnormalised inputs, wrong label mapping).

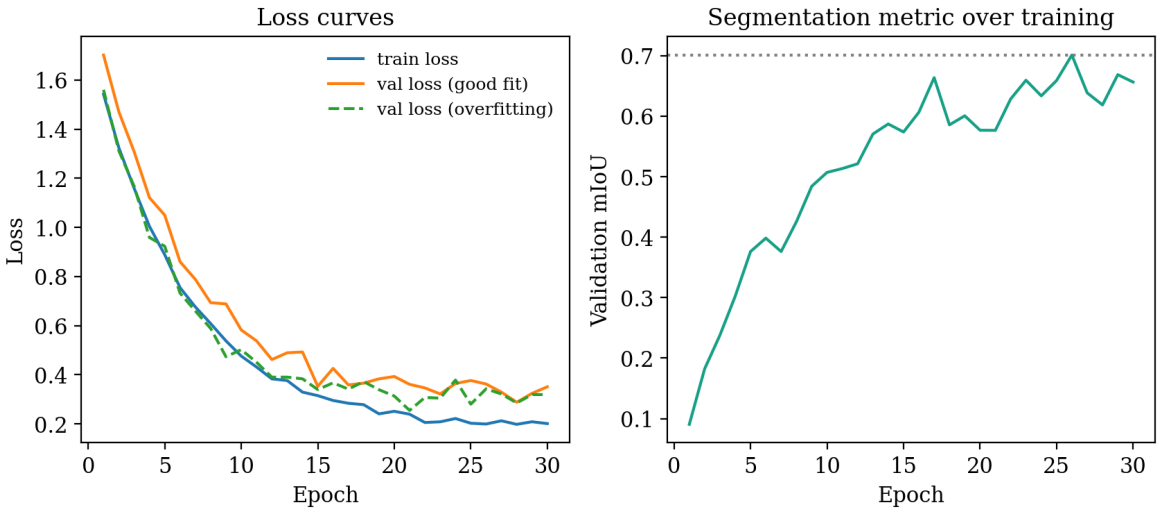


Figure 5.2: Left: loss curves. The dashed curve diverging upward is the classic overfitting signature. Right: a validation metric (here mIoU) saturating — the best epoch (dotted line) is the one to checkpoint.

5.5 Final evaluation

Reload the best checkpoint and evaluate *once* on the test set. Report overall accuracy and the per-class confusion matrix (Chapter 12); a well-trained `SimpleCNN` reaches the high-80s/low-90s on EuroSAT RGB, while a fine-tuned ResNet (Chapters 6–7) exceeds 98%. Confusions cluster among spectrally similar classes (e.g. AnnualCrop vs PermanentCrop), a hint that spatial context and more bands help.

Key Concepts

- Softmax → probabilities; cross-entropy (Eq. 5.2) is NLL of the true class; feed *logits* to `CrossEntropyLoss`.
- Five-step loop: forward, loss, backward, step, zero-grad; toggle `train()`/`eval()`.
- Checkpoint on best validation metric; touch the test set only once.
- Diverging val loss = overfitting; flat val loss = LR or data bug.

Practical Session

Notebook: `chapter_05_classification_pipeline/05_practical_eurosat_classification.ipynb`.

1. Visualise cross-entropy behaviour for varying p_y.
2. Build EuroSAT loaders with a custom `collate_fn` and normalization.
3. Train `SimpleCNN` for $\sim$15 epochs with AdamW + cosine schedule.
4. Plot loss/accuracy curves; identify the best epoch.
5. Load the best checkpoint, evaluate on test, print accuracy and confusion matrix.

Exercises & Mini-Project

5.1 LR sensitivity. Sweep learning rates $\{10^{-2}, 10^{-3}, 10^{-4}\}$; plot final accuracy and discuss the U-shape.

5.2 Class weighting. Add `weight=` to `CrossEntropyLoss` using inverse class frequency; measure the effect on the weakest class.

5.3 Early stopping. Implement patience-based early stopping; compare the selected epoch with the full-run best.

Mini-project. Wrap everything into a reusable `ClassificationTrainer` with history logging, checkpointing and early stopping; reproduce your manual result through the class API.

Chapter 6

CNN Architectures: VGG, ResNet, EfficientNet

Difficulty: Intermediate • **Duration:** ∼4 h (2 h + 2 h) • **Datasets:** EuroSAT

Learning Objectives

- Trace the evolution from LeNet/AlexNet/VGG to ResNet and EfficientNet.
- Explain the degradation problem and how residual connections solve it.
- Implement a ResNet basic block and reason about gradient flow.
- Understand EfficientNet's compound scaling and the accuracy/efficiency frontier.
- Swap backbones in a classifier and compare parameters, speed and accuracy.

6.1 A short architectural history

LeNet-5 (1998) established the `conv-pool-FC` template. AlexNet (2012) scaled it up with ReLU, dropout and GPU training, winning ImageNet by a wide margin. VGG (2014) showed that *depth* built from uniform 3×3 convolutions works well — recall (Chapter 2) that two stacked 3×3 layers match a 5×5 receptive field with fewer parameters and an extra non-linearity. But naively stacking ever more layers *degraded* accuracy, even on the training set — not overfitting, but an optimisation failure. ResNet (2015) fixed this and remains the default EO encoder.

6.2 The degradation problem and residual connections

A residual block learns a *residual* $\mathcal{F}(\mathbf{x})$ added to its input via a skip (identity) connection:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x}. \tag{6.1}$$

If the optimal mapping for a block is close to the identity, it is far easier to drive $\mathcal{F} \to 0$ than to make a stack of nonlinear layers represent the identity exactly. Crucially, the skip provides a gradient highway: by the chain rule the gradient reaching $\mathbf{x}$ contains an additive "+1" term,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(1 + \frac{\partial \mathcal{F}}{\partial \mathbf{x}}\right), \tag{6.2}$$

so gradients cannot vanish merely from depth. This lets networks train at 50, 101, even 152 layers. Figure 6.1 shows the basic block.

Deeper ResNets replace the two-3×3 block with a *bottleneck* ($1 \times 1 \to 3 \times 3 \to 1 \times 1$) that reduces and then restores channel dimension, cutting cost while keeping capacity — the building block of ResNet-50/101.

6.3 EfficientNet and compound scaling

Networks can grow along three axes: depth (layers), width (channels) and input resolution. EfficientNet (2019) showed these should be scaled *together* in fixed ratios governed by a single

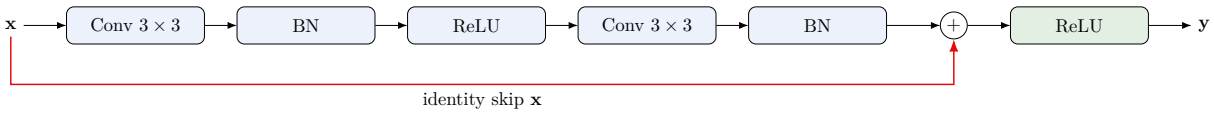


Figure 6.1: ResNet basic block (Eq. 6.1). The identity skip (red) lets the block default to a near-identity map and gives gradients an unobstructed path, enabling very deep training. A 1×1 "projection" conv replaces the skip when channel count or resolution changes.

compound coefficient ϕ:

$$\text{depth} \propto \alpha^{\phi}, \quad \text{width} \propto \beta^{\phi}, \quad \text{resolution} \propto \gamma^{\phi}, \qquad \alpha\beta^2\gamma^2 \approx 2, \tag{6.3}$$

where increasing ϕ by one roughly doubles compute. Built on mobile inverted-bottleneck (MBConv) blocks with squeeze-and-excitation, EfficientNet-B0…B7 trace an accuracy/efficiency frontier that beats ResNets at equal FLOPs — attractive when EO inference must run cheaply at scale.

6.4 Swapping backbones

In code, changing architecture is a one-line change because `torchvision` and `timm` expose a uniform API:

```
import torchvision.models as tv
m18 = tv.resnet18(weights=None, num_classes=10)
m50 = tv.resnet50(weights=None, num_classes=10)
import timm
eff = timm.create_model("efficientnet_b0", num_classes=10, in_chans=3)
```

Benchmarking on EuroSAT typically shows ResNet18 (∼11 M params) training fastest, ResNet50 (∼25 M) slightly more accurate, and EfficientNet-B0 (∼5 M) matching ResNet50 accuracy at a fraction of the parameters — the point of compound scaling. The same encoders reappear inside segmentation decoders in Chapters 9–10, which is exactly why we study them now.

Key Concepts

- VGG: depth from stacked 3×3; expensive FC head.
- Degradation $\neq$ overfitting; residual skips (Eq. 6.1) solve it and ease gradient flow.
- Bottleneck blocks scale ResNet to 50/101/152 layers efficiently.
- EfficientNet scales depth/width/resolution jointly; best accuracy per FLOP.
- The classification encoder is reused as the segmentation encoder later.

Practical Session

Notebook: `chapter_06_cnn_architectures/06_practical_cnn_architectures.ipynb`.

1. Implement a `BasicBlock` from scratch with the identity skip.
2. Visualise gradient magnitude per layer *with* and *without* the skip to see the gradient highway.
3. Tabulate parameters, forward-pass latency and accuracy for ResNet18/50 and EfficientNet-B0.
4. Train ResNet18 vs EfficientNet-B0 on EuroSAT for a few epochs and compare.

Exercises & Mini-Project

6.1 Bottleneck block. Implement the $1 \times 1 \to 3 \times 3 \to 1 \times 1$ bottleneck and verify the parameter saving versus two 3×3 layers at equal channels.

6.2 Depth ablation. Build plain (no-skip) vs residual nets of increasing depth; show that only the residual ones keep improving.

6.3 Width vs depth. At a fixed parameter budget, compare a wider-shallow and a narrower-deep network on EuroSAT.

Mini-project. Write `build_classifier(backbone, n_classes)` supporting ResNet18/50 and EfficientNet-B0, and a benchmark harness that outputs a params-vs-accuracy plot.

Chapter 7

Transfer Learning for Earth Observation

Difficulty: Intermediate • **Duration:** ∼5 h (2 h + 3 h) • **Datasets:** EuroSAT RGB + MS

Learning Objectives

- Explain why ImageNet features transfer to satellite imagery — and their limits.
- Distinguish the three transfer regimes: feature extraction, fine-tuning, from scratch.
- Adapt an RGB-pretrained first convolution to multispectral input.
- Apply discriminative (layer-wise) learning rates and progressive unfreezing.
- Quantify the accuracy and data-efficiency gains from pretraining.

7.1 Why ImageNet transfers to EO

A network pretrained on ImageNet has already learned, in its early layers, a generic visual vocabulary: oriented edges, colour-opponent filters, blobs and textures. These low- and mid-level features are *not* specific to cats and cars — they describe natural images in general, and satellite scenes share much of that statistics (edges of fields, textures of canopy and urban fabric). Reusing them gives three benefits: faster convergence, higher final accuracy, and far better performance in the small-data regime that characterises most EO labelling efforts. The transfer weakens in the deepest, most task-specific layers and across the *domain gap*: satellite imagery is nadir-looking, multispectral, and statistically different from web photos, so some adaptation is always needed.

7.2 Three transfer regimes

Let $f_\theta = h \circ g$ split a pretrained network into a feature extractor g (the convolutional backbone) and a head h.

Feature extraction (linear probe)
: freeze g, train only a new head h. Fast, few parameters, robust on tiny datasets; accuracy capped by the frozen features.

Fine-tuning
: initialise from pretrained weights and train *all* parameters at a small learning rate. Highest accuracy when you have moderate data; risk of catastrophic forgetting if the LR is too high.

From scratch
: random initialisation. Needed only with abundant labels or a large domain gap (e.g. SAR); usually worse and slower for optical EO.

On EuroSAT the ordering is reliably *fine-tune* > *linear probe* > *scratch*, and fine-tuning reaches strong accuracy in a handful of epochs where a from-scratch model needs many more.

7.3 Adapting the first convolution for multispectral input

ImageNet weights expect three input channels, but Sentinel-2 has up to 13. The first convolution's weight has shape $(C_{\text{out}}, 3, k, k)$; we must build a $(C_{\text{out}}, C_{\text{in}}, k, k)$ replacement that preserves the learned RGB filters. Two standard strategies:

1. **Copy-and-scale**: place the pretrained RGB weights on the corresponding bands and initialise the extra channels as the *mean* of the RGB filters, scaling by $3/C_{\text{in}}$ to preserve the activation magnitude.
2. **Inflate**: tile the mean RGB filter across all C_{in} channels.

```python
import torch, torch.nn as nn
def adapt_first_conv(conv_rgb: nn.Conv2d, in_ch: int) -> nn.Conv2d:
    new = nn.Conv2d(in_ch, conv_rgb.out_channels, conv_rgb.kernel_size,
                    conv_rgb.stride, conv_rgb.padding, bias=False)
    w = conv_rgb.weight.data                          # (out, 3, k, k)
    mean = w.mean(dim=1, keepdim=True)                # (out, 1, k, k)
    new.weight.data = mean.repeat(1, in_ch, 1, 1) * (3.0 / in_ch)
    new.weight.data[:, :3] = w                        # keep RGB on first 3 bands
    return new
```

This retains the value of pretraining while accepting the extra spectral bands that often lift the spectrally-confused EuroSAT classes.

7.4 Discriminative learning rates and progressive unfreezing

Early layers encode general features and should change little; later layers are task-specific and can move more. *Discriminative* (layer-wise) learning rates assign smaller LRs to earlier layer groups, e.g. $\eta_\ell = \eta_0 \, \rho^{L-\ell}$ with decay $\rho < 1$. *Progressive unfreezing* trains the head first, then unfreezes layer groups from the top down across epochs. Both reduce forgetting and stabilise fine-tuning, especially with little data.

7.5 Measuring the payoff

The clearest demonstration is a data-efficiency curve: train each regime on $\{5\%, 10\%, 25\%, 100\%\}$ of EuroSAT and plot test accuracy versus label budget. Pretrained models dominate everywhere and the gap widens as labels shrink — the practical reason transfer learning is standard in EO, where expert annotation is the bottleneck. These same pretrained encoders are dropped directly into U-Net/DeepLab decoders next, where they raise segmentation mIoU by 10–15 points (Chapters 9–10).

Key Concepts

- Early ImageNet features (edges/textures) transfer to EO; deep, domain-specific ones less so.
- Regimes: feature extraction < fine-tuning (usually best) ; scratch only with much data / large domain gap.
- Adapt the first conv to C_{in} bands while preserving RGB weights.
- Discriminative LRs and progressive unfreezing tame fine-tuning.
- Pretraining's edge is largest in the low-label regime typical of EO.

Practical Session

Notebook: `chapter_07_transfer_learning/07_practical_transfer_learning.ipynb`.

1. Train ResNet50 on EuroSAT in three regimes (scratch / linear probe / fine-tune); compare accuracy and convergence.
2. Adapt ResNet's first conv for 13-band EuroSAT-MS with `adapt_first_conv`; verify shapes and that RGB filters are preserved.
3. Apply discriminative learning rates via parameter groups.
4. Plot a data-efficiency curve over label fractions.

Exercises & Mini-Project

7.1 Layer-wise LR decay. Implement $\eta_\ell = \eta_0 \rho^{L-\ell}$ with parameter groups; sweep ρ and report the best.

7.2 MS vs RGB. Compare fine-tuned accuracy on EuroSAT-RGB vs MS; which classes benefit most from extra bands?

7.3 Forgetting. Fine-tune at a deliberately high LR and show catastrophic forgetting in the loss curve; fix it with a warmup.

Mini-project. Build a `ProgressiveFinetuner` that unfreezes layer groups on a schedule, and reproduce or beat your best fine-tuning result.

Chapter 8

Data Augmentation for Satellite Imagery

Difficulty: Intermediate • **Duration:** ∼4 h (1.5 h + 2.5 h) • **Datasets:** EuroSAT; LoveDA

Learning Objectives

- Build geometric and photometric augmentation pipelines with `albumentations`.
- Identify which augmentations are label-safe for satellite imagery and which are not.
- Apply joint image+mask augmentation for segmentation.
- Understand and implement MixUp and CutMix.
- Empirically measure the accuracy gain from augmentation via an ablation.

Augmentation is the cheapest and most reliable regulariser for vision models: it synthesises new, label-consistent training examples on the fly, expanding the effective dataset and encoding invariances we know the task should have. Satellite imagery has a special property that makes some augmentations *more* valid than for natural photos — and others invalid.

8.1 Why augmentation is especially powerful in EO

Nadir-looking satellite scenes have no canonical "up". A forest rotated 90° or mirrored is still a forest, so the full dihedral group of flips and 90°-rotations is label-preserving — eight free views of every patch, an invariance natural photos do not enjoy. Combined with the chronic scarcity of labels in EO, this makes geometric augmentation unusually effective. Figure 8.1 shows representative transforms.

8.2 A pipeline with albumentations

`albumentations` is fast, supports many bands, and — crucially for segmentation — transforms image and mask jointly. A typical classification pipeline:

```
import albumentations as A
train_tf = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomRotate90(p=0.5),
    A.RandomBrightnessContrast(0.2, 0.2, p=0.5),
    A.GaussNoise(var_limit=(10, 50), p=0.3),
    A.Normalize(mean=MEAN, std=STD),
])
```

Validation/test pipelines contain *only* normalization — never randomise the data you measure on.

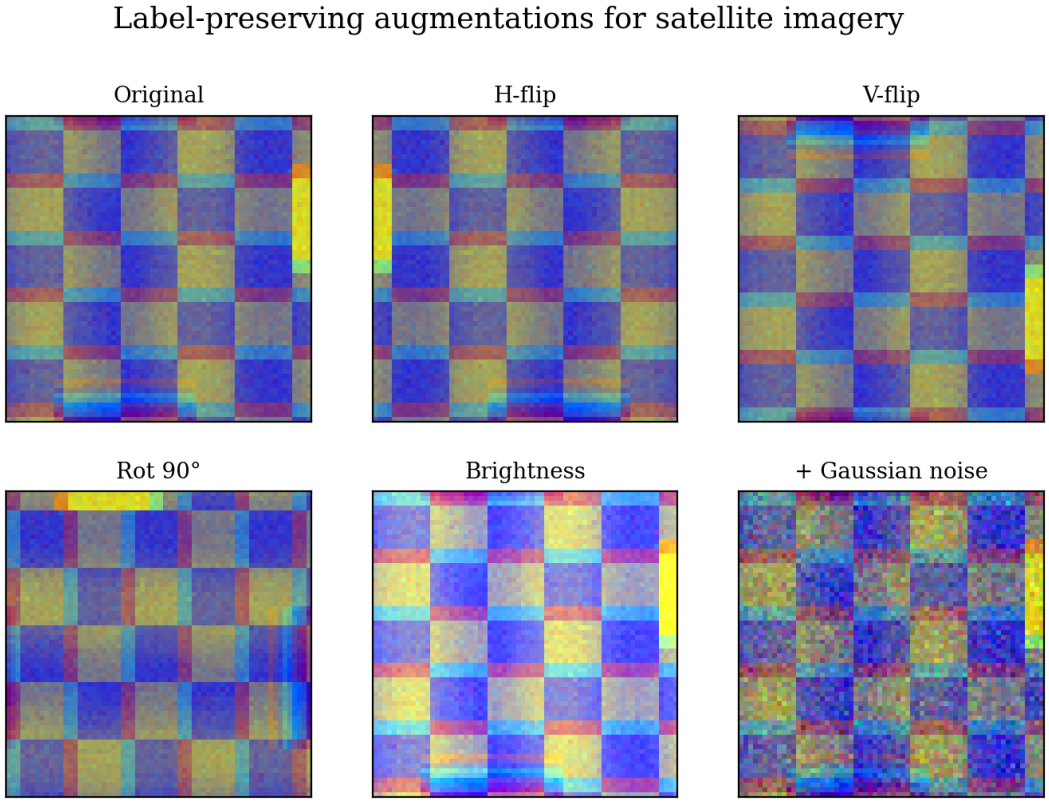


Figure 8.1: Label-preserving augmentations. Flips and 90° rotations are exactly valid for nadir imagery; brightness/contrast jitter and mild Gaussian noise simulate illumination and sensor variation.

What to avoid. Augmentations that break physical meaning hurt EO models: large hue shifts destroy the spectral signatures that distinguish materials; heavy elastic/perspective warps fabricate impossible geometry; aggressive blur erases the texture cues the network relies on. Prefer mild, physically plausible photometric jitter and rely on the free geometric invariances.

8.3 Joint image–mask augmentation

For segmentation, every geometric transform applied to the image *must* be applied identically to the label mask; photometric transforms apply to the image only. `albumentations` handles this when you pass both:

```
aug = A.Compose([A.HorizontalFlip(p=0.5), A.RandomRotate90(p=0.5),
                 A.RandomBrightnessContrast(p=0.5)])
out = aug(image=img, mask=mask)          # geometric ops applied to both
img_a, mask_a = out["image"], out["mask"]
```

Use nearest-neighbour interpolation for masks (labels are categorical, never interpolate class indices) and bilinear for images.

8.4 MixUp and CutMix

These mix *pairs* of examples and their labels, regularising decision boundaries. *MixUp* blends two samples linearly:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda)y_j, \qquad \lambda \sim \mathrm{Beta}(\alpha, \alpha). \tag{8.1}$$

CutMix instead pastes a rectangular patch of one image into another and sets the label mixing coefficient to the patch's area fraction. Both improve calibration and robustness for classification; for segmentation, CutMix-style region pasting is more natural than pixel blending. Use them as a complement to, not a replacement for, geometric augmentation.

8.5 Proving it works: an ablation

Augmentation's value should be measured, not assumed. Train the same model for a few epochs under increasing augmentation (none → geometric → geometric+photometric → +MixUp) and plot validation accuracy/mIoU. On EO data the geometric step alone typically yields a clear gain, and the gain grows as the labelled set shrinks — the same low-data story as transfer learning.

Key Concepts

- Nadir imagery ⇒ flips + 90° rotations are exactly label-preserving (8 free views).
- Only normalize at validation/test time; never randomise evaluation data.
- Avoid hue shifts/heavy warps/blur that destroy spectral or texture cues.
- Segmentation: apply geometric ops to image *and* mask (nearest-neighbour for masks).
- MixUp/CutMix mix samples and labels; complement geometric augmentation.

Practical Session

Notebook: `chapter_08_augmentation/08_practical_augmentation_pipeline.ipynb`.

1. Showcase each transform on one EuroSAT image in a grid.
2. Build an EO-aware `albumentations` pipeline; visualise eight random draws of one patch.
3. Demonstrate joint image+mask augmentation on a LoveDA sample.
4. Implement MixUp and visualise a blended pair with its soft label.
5. Run a 3-epoch ablation (no-aug vs aug) and plot the accuracy gain.

Exercises & Mini-Project

8.1 Policy search. Grid over augmentation strengths; find the policy maximising validation accuracy without underfitting.

8.2 Custom EO transform. Implement a band-wise reflectance-jitter transform that perturbs each band independently within physical bounds.

8.3 Mask safety. Construct a case where bilinear mask interpolation creates invalid class indices; fix it with nearest-neighbour.

Mini-project. Build `get_train_transforms(task, bands)` returning classification and segmentation pipelines (RGB and multispectral variants) and document each choice.

Part III

Segmentation and Geospatial Pipelines

Chapter 9

Semantic Segmentation and the U-Net

Difficulty: Intermediate → Advanced • **Duration:** ~6 h (2.5 h + 3.5 h) (torchgeo) • **Datasets:** LoveDA

Learning Objectives

- Reframe the task from one label per image to one label per pixel.
- Understand the encoder–decoder design and the role of skip connections.
- Implement a U-Net from scratch and trace its tensor shapes.
- Choose and combine segmentation losses (cross-entropy, Dice).
- Train and evaluate a U-Net on the LoveDA land-cover dataset.

9.1 From classification to dense prediction

Classification answers "what is in this patch?" with a single label. *Semantic segmentation* answers "what is at every pixel?", producing an output of shape (K, H, W) — a per-pixel class distribution at full input resolution. The challenge is that the classification encoder repeatedly *downsamples* (for receptive field and efficiency), so we must recover spatial detail. The encoder–decoder family does this: an encoder contracts to a compact, semantically rich representation, and a decoder expands back to full resolution. The decoder upsamples with transposed convolutions or interpolation+convolution; the key difficulty is that pooling has discarded the fine localisation information the decoder needs.

9.2 The U-Net architecture

U-Net (Ronneberger et al., 2015) solves this with *skip connections* that carry high-resolution encoder feature maps directly to the matching decoder stage, concatenating them before each decoder convolution (Figure 9.1). The encoder provides "what"; the skips restore "where". This single idea is why U-Net dominates medical and EO segmentation.

A minimal from-scratch U-Net uses a reusable double-convolution block:

```
import torch, torch.nn as nn
class DoubleConv(nn.Module):
    def __init__(self, cin, cout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(cin, cout, 3, padding=1, bias=False),
            nn.BatchNorm2d(cout), nn.ReLU(inplace=True),
            nn.Conv2d(cout, cout, 3, padding=1, bias=False),
            nn.BatchNorm2d(cout), nn.ReLU(inplace=True))
    def forward(self, x): return self.net(x)

class Up(nn.Module):
```

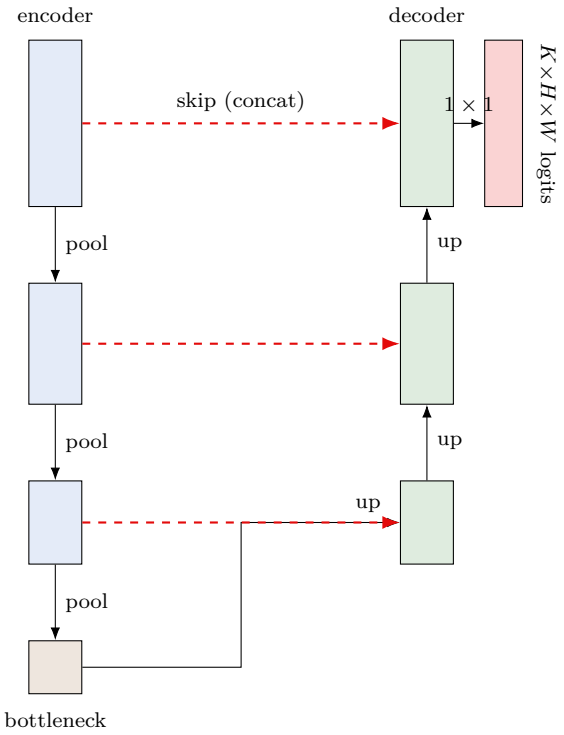


Figure 9.1: U-Net. The encoder (blue) halves resolution and doubles channels at each stage; the decoder (green) mirrors it, upsampling and halving channels. Dashed red skip connections concatenate same-resolution encoder features into the decoder, restoring spatial detail lost to pooling. A final 1×1 convolution maps to K class logits per pixel.

```
    def __init__(self, cin, cout):
        super().__init__()
        self.up = nn.ConvTranspose2d(cin, cin // 2, 2, stride=2)
        self.conv = DoubleConv(cin, cout)            # cin = up + skip channels
    def forward(self, x, skip):
        x = self.up(x)
        x = torch.cat([skip, x], dim=1)              # concatenate skip
        return self.conv(x)
```

Tracing shapes for a 256×256 input with base width 64: the encoder produces feature maps at $256, 128, 64, 32$ resolutions with $64, 128, 256, 512$ channels; the decoder reverses this, each `Up` consuming the matching skip; the head outputs $(K, 256, 256)$.

9.3 Loss functions for segmentation

Per-pixel cross-entropy (Eq. 5.2, summed over pixels) is the baseline, but land-cover classes are imbalanced (water and bare soil are rare), and CE is dominated by frequent classes. The *Dice loss* directly optimises overlap and is robust to imbalance. For predicted soft probabilities p and one-hot target g over N pixels and class k:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{K} \sum_{k=1}^{K} \frac{2 \sum_i p_{ik}\, g_{ik} + \varepsilon}{\sum_i p_{ik} + \sum_i g_{ik} + \varepsilon}. \tag{9.1}$$

In practice a *combined* loss works best, pairing CE's well-behaved gradients with Dice's overlap objective:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda\, \mathcal{L}_{\text{Dice}}, \qquad \lambda \approx 1. \tag{9.2}$$

For severe imbalance, class-weighted CE or focal loss (down-weighting easy pixels) are alternatives. We measure success with IoU/mIoU, defined in Chapter 12.

9.4 Training on LoveDA

LoveDA (Wang et al., 2021) provides 5,987 high-resolution (1024×1024) scenes over urban and rural domains with seven land-cover classes, auto-downloaded via `torchgeo`. The recipe combines everything so far: tile to 512×512 (Chapter 4), augment image+mask jointly (Chapter 8), use a U-Net (optionally with a pretrained encoder, Chapter 7), optimise the combined loss, and checkpoint on validation mIoU. The ablation that most convinces students: train the U-Net *with* and *without* skip connections — the no-skip variant produces blurry, mislocalised masks and markedly lower mIoU, making the role of the skips concrete.

Key Concepts

- Segmentation outputs (K, H, W): a class distribution per pixel.
- Encoder–decoder recovers resolution; skip connections restore localisation ("what" + "where").
- Decoder upsamples via transposed conv (or interpolate+conv) and concatenates skips.
- Combine cross-entropy with Dice (Eq. 9.1) for imbalanced land cover.
- Pretrained encoders and joint image+mask augmentation carry straight over from earlier chapters.

Practical Session

Notebook: `chapter_09_semantic_segmentation/09_practical_unet_segmentation.ipynb`.

1. Implement `DoubleConv`, `Down`, `Up`, `OutConv` and assemble a U-Net; print feature-map shapes at every stage.
2. Implement Dice loss and the combined CE+Dice loss.
3. Load LoveDA via `torchgeo`; build a segmentation `DataLoader` with joint augmentation; visualise image/mask pairs.
4. Train for several epochs; plot loss and validation mIoU curves.
5. Load the best model and display RGB | GT | prediction triplets.

Exercises & Mini-Project

9.1 No-skip ablation. Remove the skip connections; quantify the mIoU drop and show the blurrier masks.

9.2 Loss comparison. Train with CE only, Dice only, and CE+Dice; compare per-class IoU, especially on rare classes.

9.3 Depth/width. Vary U-Net depth and base width; relate capacity to mIoU and memory.

Mini-project. Add an ImageNet-pretrained ResNet encoder to your U-Net (manual or via SMP, Chapter 10) and measure the mIoU gain over the from-scratch version.

Chapter 10

Advanced Segmentation Architectures

Difficulty: Advanced • **Duration:** ∼5 h (2 h + 3 h) • **Datasets:** LoveDA

Learning Objectives

- Use `segmentation-models-pytorch` (SMP) to build decoders on pretrained encoders.
- Understand atrous (dilated) convolution and DeepLabV3+'s ASPP module.
- Understand multi-scale context: FPN and PSPNet pyramid pooling.
- Benchmark architectures fairly on a common dataset and budget.
- Choose an architecture/encoder for a given EO constraint.

The hand-built U-Net of Chapter 9 taught the mechanics. In practice we assemble decoders on top of pretrained encoders with a library, and choose among architectures designed to capture *multi-scale context* — essential in EO, where land-cover objects range from single trees to whole agricultural regions.

10.1 segmentation-models-pytorch

SMP exposes every architecture through one uniform constructor, pairing a decoder with any of dozens of pretrained encoders (`timm`-backed):

```
import segmentation_models_pytorch as smp
model = smp.Unet(encoder_name="resnet50", encoder_weights="imagenet",
                 in_channels=3, classes=7)
# swap decoder by changing the class: smp.DeepLabV3Plus / smp.FPN / smp.PSPNet
```

For multispectral input set `in_channels=13`; SMP adapts the first convolution automatically (cf. Chapter 7). This one-line flexibility is what makes principled architecture comparison feasible.

10.2 Atrous convolution and DeepLabV3+

A standard convolution must pool to enlarge its receptive field, sacrificing resolution. *Atrous* (dilated) convolution inserts gaps of rate r between kernel taps, enlarging the receptive field *without* downsampling or extra parameters. A dilated $k \times k$ kernel has an effective size

$$k_{\text{eff}} = k + (k-1)(r-1), \tag{10.1}$$

so a 3×3 kernel at rate r=6 covers 13×13. DeepLabV3+ uses *Atrous Spatial Pyramid Pooling* (ASPP): several parallel atrous convolutions at different rates (e.g. $6, 12, 18$) plus image-level global pooling, concatenated to fuse context at multiple scales, followed by a light decoder that recovers boundaries. It excels when objects span very different sizes — the norm in land cover.

10.3 FPN and PSPNet

Feature Pyramid Networks (FPN) build a top-down pathway with lateral connections, producing semantically strong features at every resolution and fusing predictions across the pyramid —

naturally multi-scale and efficient. *PSPNet* introduces the *pyramid pooling module*, pooling the deepest feature map at several grid scales ($1 \times 1, 2 \times 2, 3 \times 3, 6 \times 6$), processing each, and concatenating, injecting global scene context that helps disambiguate locally-similar classes (e.g. shadowed roof vs water). All three — DeepLabV3+, FPN, PSPNet — attack the same problem (context at multiple scales) by different means.

10.4 A fair benchmark

Comparisons are only meaningful under a fixed protocol: same encoder, same data splits, same augmentation, same epochs and optimiser, varying only the decoder. Report mIoU, parameters, and inference latency.

Decoder	Context mechanism	Params	Strength	Typical use
U-Net	skip connections	low–med	sharp boundaries	default, small data
FPN	feature pyramid	low	multi-scale, fast	efficient inference
PSPNet	pyramid pooling	med	global context	scene-level classes
DeepLabV3+	ASPP (atrous)	med–high	multi-scale + edges	best all-round

On LoveDA with a ResNet50 encoder the four are usually within a few mIoU points; DeepLabV3+ tends to lead on boundary-heavy urban scenes, U-Net remains a strong, low-memory default, and FPN is attractive when latency matters. The lesson is to *measure on your data and budget* rather than trust leaderboard ranks from another domain.

Key Concepts

- SMP = decoder × pretrained encoder via one constructor; trivial multispectral support.
- Atrous convolution grows the receptive field without losing resolution; $k_{\text{eff}} = k + (k-1)(r-1)$.
- DeepLabV3+ (ASPP), FPN and PSPNet all capture multi-scale context differently.
- Benchmark fairly: fix everything but the decoder; report mIoU, params, latency.
- No universal winner — choose for your data, objects' scale, and compute budget.

Practical Session

Notebook: `chapter_10_advanced_segmentation/10_practical_advanced_segmentation.ipynb`.

1. Build U-Net, DeepLabV3+, FPN and PSPNet with SMP on a shared ResNet50 encoder; print parameter counts.
2. Visualise how atrous rate enlarges the receptive field.
3. Run a fixed-protocol benchmark of the four decoders for a few epochs.
4. Plot mIoU vs params vs latency; pick a winner.
5. Train the winner longer and display prediction triplets.

Exercises & Mini-Project

10.1 Encoder swap. Hold the decoder fixed and compare ResNet34/50 and EfficientNet-B0/B4 encoders by mIoU and speed.

10.2 ASPP rates. Vary DeepLab's atrous rates; relate them to the object scales in LoveDA.

10.3 Multispectral. Rebuild the best model with `in_channels=13` on a multispectral dataset; measure the gain over RGB.

Mini-project. Produce a one-page "architecture selection" report recommending a decoder+encoder for (a) a latency-constrained edge device and (b) a maximum-accuracy server, justified by your benchmark.

Chapter 11

The Full EO Pipeline: Tiling, Inference, GeoTIFF

Difficulty: Advanced • **Duration:** ∼6 h (2 h + 4 h) • **Datasets:** Sentinel-2 scenes (synthetic + real)

Learning Objectives

- Assemble a complete pipeline: raw scene → tiles → inference → map.
- Implement memory-safe sliding-window inference over a full Sentinel-2 tile.
- Blend overlapping predictions with Gaussian weighting to remove seams.
- Export a georeferenced GeoTIFF with correct CRS and transform.
- Compute per-class area statistics from a prediction map.

This chapter closes the loop: a trained segmentation model becomes a deployable system that turns a ∼10980 × 10980 Sentinel-2 scene into a georeferenced land-cover map. No single GPU can hold such a scene, so inference must be tiled, and the tiles' predictions stitched back into one seamless raster.

11.1 Pipeline overview

The end-to-end flow chains the components built in earlier chapters (Figure 11.1): load and preprocess the scene (Chapter 4), tile it with overlap, run the model on each tile (Chapters 9–10), blend predictions, reconstruct the full map, and write a GeoTIFF.

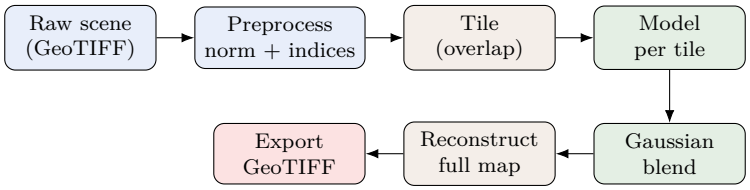


Figure 11.1: The full inference pipeline. Every stage corresponds to a tool built in an earlier chapter; only the orchestration is new.

11.2 Sliding-window inference

We slide a $T \times T$ window with stride $s < T$ (overlap $T - s$) across the scene. For each window we run the model to obtain class *probabilities* (not hard labels — we need to average), and accumulate them into a full-scene buffer $P \in \mathbb{R}^{K \times H \times W}$ together with a weight buffer A. The final per-pixel prediction is $\arg\max_k P[k]/A$.

```
import numpy as np, torch
def sliding_window(model, scene, T=512, s=384, K=7, device="cuda"):
    C, H, W = scene.shape
```

```
    P = np.zeros((K, H, W), np.float32); A = np.zeros((H, W), np.float32)
    wmap = gaussian_weight(T)                       # (T, T), peaks at centre
    model.eval()
    with torch.no_grad():
        for y in range(0, H - T + 1, s):
            for x in range(0, W - T + 1, s):
                tile = torch.from_numpy(scene[:, y:y+T, x:x+T])[None].to(device)
                prob = model(tile).softmax(1)[0].cpu().numpy()   # (K, T, T)
                P[:, y:y+T, x:x+T] += prob * wmap
                A[y:y+T, x:x+T]    += wmap
    return (P / np.maximum(A, 1e-6)).argmax(0)      # (H, W) labels
```

Edge handling (padding the last partial tile) and batching multiple tiles together for throughput are the practical refinements.

11.3 Why Gaussian blending

If overlapping tiles are averaged with equal weight, predictions near a tile's border — where the receptive field is truncated and context is missing — are unreliable and produce visible grid *seams*. Weighting each tile by a 2D Gaussian $w(u, v) \propto \exp\!\left(-\frac{(u-c)^2+(v-c)^2}{2\sigma^2}\right)$ centred on the tile (so centre pixels dominate, borders fade) makes the contributions of adjacent tiles transition smoothly, eliminating seams. The ablation "equal weight vs Gaussian" makes the artefact and its cure obvious.

11.4 Georeferenced export

A prediction is useless to a GIS analyst unless it carries its geolocation. We write the label raster with the *same* CRS and affine transform as the input scene, so it overlays exactly in QGIS:

```
import rasterio
with rasterio.open(src_path) as src:
    profile = src.profile
profile.update(count=1, dtype="uint8", nodata=0)
with rasterio.open("landcover.tif", "w", **profile) as dst:
    dst.write(pred.astype("uint8"), 1)
    dst.write_colormap(1, {i: hex_to_rgb(c) for i, c in enumerate(PALETTE)})
```

Embedding a colour map renders the classes correctly without manual styling.

11.5 Area statistics

Because each pixel covers a known ground area ($10 \times 10\,\mathrm{m} = 100\,\mathrm{m}^2$ for Sentinel-2 10 m bands), counting pixels per class converts directly to hectares:

$$\mathrm{area}_k\,[\mathrm{ha}] = \frac{n_k \cdot A_{\mathrm{px}}}{10^4}, \qquad A_{\mathrm{px}} = |a \cdot e|\ \mathrm{m}^2, \tag{11.1}$$

turning a colourful map into the quantitative output stakeholders actually want ("X hectares of forest, Y of built-up"). Figure 11.2 shows a representative scene and its predicted land-cover map.

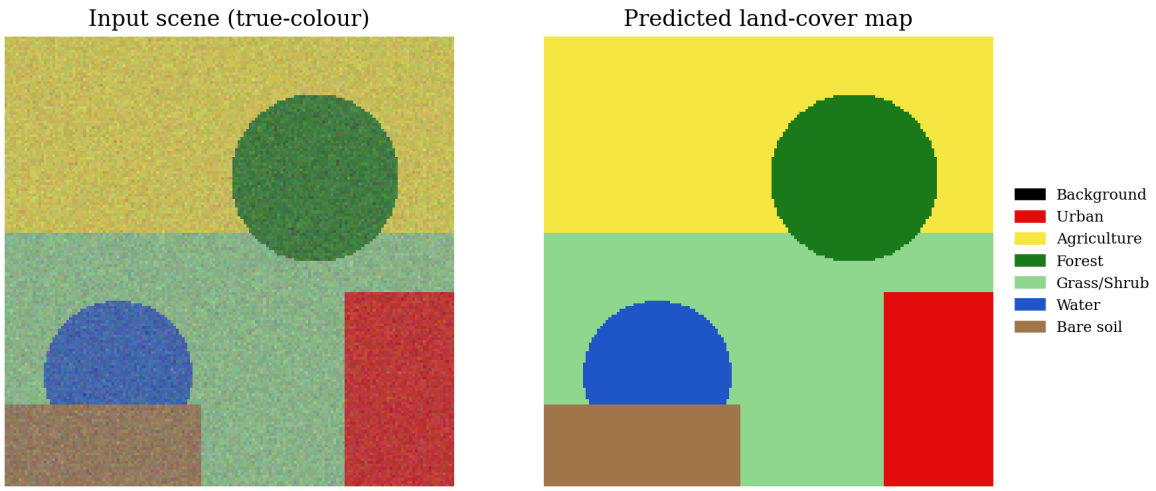


Figure 11.2: Input scene (left) and reconstructed land-cover prediction (right) with a class legend — the deliverable of the full pipeline and of the capstone.

Key Concepts

- Full-scene inference is tiled; accumulate *probabilities*, then argmax.
- Overlap + Gaussian blending removes tile seams from truncated-context borders.
- Export with the source CRS/transform (and a colour map) for GIS-ready GeoTIFFs.
- Pixel area → hectares gives quantitative, decision-ready statistics.
- The pipeline is pure orchestration of tools from Chapters 4–10.

Practical Session

Notebook: `chapter_11_full_eo_pipeline/11_practical_end_to_end_pipeline.ipynb`.

1. Create (or load) a large multi-band scene with spatial structure.
2. Implement `gaussian_weight` and the sliding-window inference loop.
3. Run inference with a trained (or demo) model; reconstruct the full map.
4. Compare reconstructions *with* and *without* Gaussian blending.
5. Export a georeferenced GeoTIFF with a colour map; compute per-class hectares.

Exercises & Mini-Project

11.1 Overlap effect. Sweep stride/overlap; quantify seam artefacts and runtime trade-offs.

11.2 Batched tiles. Batch multiple tiles per forward pass; measure the throughput gain.

11.3 QGIS check. Open your GeoTIFF in QGIS over an OSM basemap; verify alignment and discuss any offset.

Mini-project. Wrap the pipeline into `predict_scene(model, path) -> geotiff` with a CLI, then run it on a real downloaded Sentinel-2 tile.

Chapter 12

Evaluation and Interpretability

Difficulty: Advanced • **Duration:** ~4 h (2 h + 2 h) • **Datasets:** LoveDA; trained checkpoints

Learning Objectives

- Define and compute IoU, mIoU, Dice, pixel accuracy and per-class F1.
- Build and correctly normalise a confusion matrix for segmentation.
- Apply GradCAM to see where a CNN "looks".
- Use prediction-entropy maps to flag uncertain regions.
- Conduct a structured failure analysis and propose targeted fixes.

A land-cover map is only as trustworthy as its evaluation. This chapter formalises the metrics used throughout the course and adds interpretability tools that turn "the mIoU is 0.58" into actionable understanding of *where* and *why* the model fails.

12.1 Segmentation metrics

Let TP, FP, FN be the per-class pixel counts. The *Intersection over Union* (Jaccard index) for class k and its mean over classes are

$$\mathrm{IoU}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}, \qquad \mathrm{mIoU} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{IoU}_k. \tag{12.1}$$

mIoU is the standard segmentation headline because, unlike pixel accuracy, it weights every class equally and is not inflated by a dominant background. The *Dice* (F1) coefficient is closely related,

$$\mathrm{Dice}_k = \frac{2\,\mathrm{TP}_k}{2\,\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k} = \frac{2\,\mathrm{IoU}_k}{1 + \mathrm{IoU}_k}, \tag{12.2}$$

always at least as large as IoU and more forgiving of small errors (Figure 12.1). *Pixel accuracy* (fraction of correctly labelled pixels) is reported for completeness but is misleading under class imbalance.

12.2 The confusion matrix

The $K \times K$ confusion matrix C, with C_{ij} the number of pixels of true class i predicted as j, is the richest single diagnostic. *Row-normalise* it ($C_{ij} / \sum_j C_{ij}$) to read per-class recall and to see which classes are confused for which (Figure 12.2). In land cover the off-diagonal mass concentrates among semantically adjacent classes — grassland vs agriculture, urban vs bare soil — which points directly at where extra bands, indices or augmentation would help.

12.3 GradCAM: what the model sees

GradCAM produces a coarse heat-map of the image regions most responsible for a prediction. For target class c and the activations A^k of a chosen convolutional layer, the neuron-importance

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \qquad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

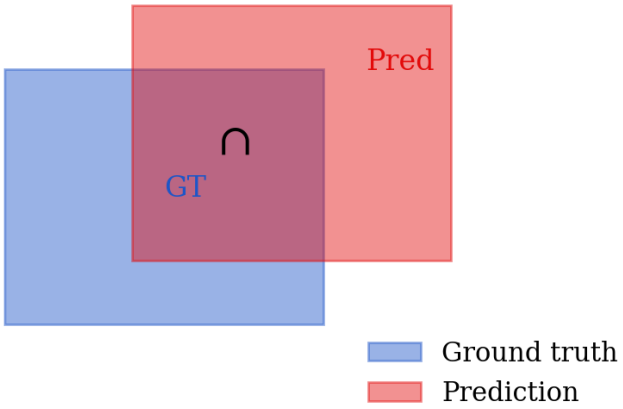


Figure 12.1: IoU and Dice for a predicted region (red) versus ground truth (blue). IoU divides intersection by union; Dice doubles the intersection over the sum of areas. Both reward overlap and penalise both false positives and false negatives.

weights are the gradient-global-average

$$\alpha_k^c = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{GradCAM}} = \text{ReLU}\Big(\sum_k \alpha_k^c A^k\Big), \tag{12.3}$$

upsampled to the input size. For EO it answers questions like "did the network classify this patch as Industrial because of the buildings or because of an adjacent road?", exposing spurious correlations (a model keying on image borders or compression artefacts is a real failure mode).

12.4 Uncertainty via entropy

The per-pixel predictive entropy of the softmax distribution,

$$\mathcal{H}(\mathbf{p}) = -\sum_{k=1}^{K} p_k \log p_k, \tag{12.4}$$

is a cheap uncertainty proxy: high entropy marks pixels where the model hedges — typically class boundaries and genuinely ambiguous land cover. An entropy *map* overlaid on the prediction shows reviewers exactly where to look and is the basis of active-learning sample selection.

12.5 Failure analysis

Good practice closes the loop: rank validation tiles by error, inspect the worst, categorise the mistakes (boundary leakage, whole-class confusion, rare-class misses, tiling seams), attribute each to a cause, and propose a targeted fix (class weighting for rare classes, more overlap for seams, extra indices for spectral confusion). A short, honest discussion of failure cases is what separates a portfolio project from a toy — and is explicitly rewarded in the capstone rubric.

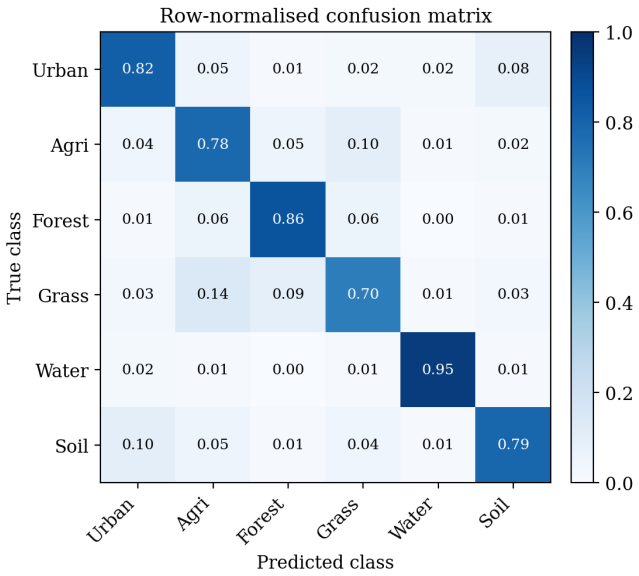


Figure 12.2: Row-normalised confusion matrix. The diagonal is per-class recall; bright off-diagonal cells (e.g. grassland → agriculture) reveal systematic, explainable confusions to target.

Key Concepts

- mIoU (Eq. 12.1) is the class-balanced headline; pixel accuracy misleads under imbalance.
- Dice $\geq$ IoU; both penalise FP and FN.
- Row-normalised confusion matrices reveal *which* classes are confused.
- GradCAM localises evidence; entropy maps localise uncertainty.
- Structured failure analysis turns metrics into targeted improvements.

Practical Session

Notebook: `chapter_12_evaluation_interpretability/12_practical_evaluation_interpretability.ipynb`.

1. Implement per-class IoU, mIoU, Dice, pixel accuracy and F1 from a confusion matrix.
2. Plot a publication-quality row-normalised confusion matrix.
3. Apply GradCAM to a trained model for selected classes; interpret the heat-maps.
4. Compute and visualise a prediction-entropy map.
5. Rank tiles by error and write a short failure-mode report.

Exercises & Mini-Project

12.1 IoU vs Dice. Construct prediction/GT pairs where IoU and Dice disagree most; explain the geometry.

12.2 Boundary metric. Implement a boundary-IoU that scores only pixels near class edges; compare with standard IoU.

12.3 Calibration. Plot a reliability diagram of predicted confidence vs accuracy; is the model over- or under-confident?

Mini-project. Build an `evaluation_report(model, loader)` that emits a metrics table, confusion matrix, GradCAM panel, entropy map and a ranked failure gallery as a single figure/markdown bundle.

Part IV

Capstone and Reference

Chapter 13

Capstone: End-to-End Land-Cover Mapping

Difficulty: Advanced • **Duration:** ∼10 h (∼4 weeks part-time) • **Datasets:** LoveDA and/or Sentinel-2 + OSM labels

Learning Objectives

- Integrate the entire course into one production-grade system, independently.
- Take a raw Sentinel-2 Level-2A scene to a georeferenced land-cover GeoTIFF.
- Justify every design choice (encoder, decoder, loss, augmentation).
- Evaluate rigorously and analyse failures.
- Package a portfolio-ready deliverable.

This is not a tutorial: there is no step-by-step guide. You apply everything from Chapters 1–12 to solve a real mapping problem on your own. Notebook: `notebooks/capstone/capstone_land_cover_mapping.ipynb`.

13.1 Problem statement

Given a Sentinel-2 Level-2A tile (bottom-of-atmosphere reflectance, 12 usable bands, ∼110 × 110 km), produce a pixel-level land-cover map over seven classes.

ID	Class	Colour
0	Background / no-data	`#000000`
1	Urban / built-up	`#E30B0B`
2	Agriculture	`#F5E642`
3	Forest	`#1A7A1A`
4	Shrubland / grassland	`#8FD68F`
5	Water	`#1E56C8`
6	Bare soil / rock	`#A0754A`

13.2 Mandatory components

Data pipeline
load with `rasterio`/`rioxarray`; stack B02/B03/B04/B08 (10 m) with B05–B07/B8A/B11/B12 (20 m, upsampled); compute ≥2 indices (NDVI, NDWI, . . .); normalize with training statistics; generate 512 × 512 tiles with configurable overlap.

Model
an SMP model with a pretrained encoder; justify the encoder (ResNet50, EfficientNet-B4 or MobileNetV3) and the decoder (U-Net, FPN or DeepLabV3+); handle class imbalance with weighted Dice or focal loss.

Training
LoveDA or Sentinel-2 + OSM-derived labels; early stopping; ≥4 augmentation types; log loss and IoU per epoch; checkpoint on best validation mIoU.

Evaluation
per-class IoU, mIoU, F1, overall accuracy; confusion matrix; ≥5 qualitative examples with GT overlays; GradCAM for ≥1 class.

Inference
sliding-window over the full tile with Gaussian blending; reconstruct; export a georeferenced GeoTIFF with correct CRS/transform.

Visualisation
colour-coded map with legend; true-colour vs prediction side-by-side; per-class area in hectares.

13.3 Deliverables and rubric

Deliverable	Format	Weight
End-to-end executed notebook	`.ipynb`	40%
Trained model checkpoint	`.pth`	10%
Prediction GeoTIFF (with CRS)	`.tif`	15%
Evaluation report (metrics + confusion matrix)	MD/PDF	20%
Technical writeup (max 2 pages)	PDF/MD	15%

The 100-point rubric is split evenly across five areas — Data pipeline, Model architecture, Training, Evaluation, Inference+GeoTIFF (20 points each). Full marks require not just working code but *justified* choices: a silently used default encoder scores zero on that criterion even if accurate. A validation mIoU $\geq$ 40% earns the evaluation points; 30–40% partial. Grades: Distinction 85–100, Merit 70–84, Pass 55–69.

Week	Focus
1	Data pipeline: loading, normalization, tiling
2	Model setup, first training run, baseline metrics
3	Ablations (loss functions, encoders, augmentation)
4	Inference pipeline, GeoTIFF export, evaluation report

13.4 Extension ideas

For a Distinction-level project, push beyond the brief: multi-temporal fusion (stack two seasons to improve agricultural classes); domain generalisation (train on one region, test on another, analyse the shift); Sentinel-1 SAR fusion (add backscatter channels, measure gains on water/urban); uncertainty via Monte-Carlo dropout; CRF post-processing for sharper boundaries; or a Streamlit/FastAPI demo that ingests a GeoTIFF and returns a prediction map.

13.5 Portfolio checklist

- Notebook runs end-to-end from a clean kernel restart.
- Every cell has explanatory markdown; every figure has titles, labels, legends.
- GeoTIFF opens and aligns correctly in QGIS.
- A `README` explains how to reproduce; checkpoint < 500 MB (Git LFS if needed).

- The report discusses failure cases honestly.

The point of the capstone

The capstone is the artefact you show an employer. A clear writeup, a banner figure (true-colour vs prediction), an honest limitations section and a one-line headline ("ResNet50 + DeepLabV3+, mIoU 58% on LoveDA") demonstrate end-to-end competence more convincingly than any single notebook from the course.

Chapter A

Practical Infrastructure

This appendix collects the software, environment and hardware details needed to run every notebook in the course.

A.1 Software stack

Layer	Library	Version
Language	Python	3.12
Deep learning	PyTorch	≥2.2
Computer vision	torchvision	≥0.17
Segmentation	segmentation-models-pytorch	≥0.3.3
Metrics	torchmetrics	≥1.3
EO datasets	torchgeo	≥0.6
Raster I/O	rasterio	≥1.3
Raster + xarray	rioxarray	≥0.15
Multi-dim arrays	xarray	≥2024.1
Vector + CRS	geopandas	≥0.14
Augmentation	albumentations	≥1.4
Visualisation	matplotlib / seaborn	latest
Interpretability	pytorch-grad-cam	≥1.5
Notebooks	jupyterlab	≥4.1

A.2 Environment setup

uv (recommended).

```
curl -LsSf https://astral.sh/uv/install.sh | sh
git clone <repo-url> CNN_projects && cd CNN_projects
uv sync                 # installs from pyproject.toml into .venv
uv run jupyter lab
```

conda or pip.

```
conda env create -f environment.yml && conda activate eo-cnn && pip install -e .
# or
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
```

Google Colab. Every notebook begins with a setup cell that installs dependencies, mounts Drive, and detects the device:

```
IN_COLAB = 'google.colab' in str(get_ipython())
if IN_COLAB:
    !pip install -q torch torchvision torchgeo segmentation-models-pytorch \
```

```
    torchmetrics albumentations rasterio rioxarray geopandas grad-cam tqdm
    from google.colab import drive; drive.mount('/content/drive')
    DATA_ROOT = '/content/drive/MyDrive/eo_cnn_data'
else:
    DATA_ROOT = '../data'
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
FAST_MODE = not torch.cuda.is_available()   # reduces epochs/data on CPU
```

A.3 Repository structure

```
CNN_projects/
|- README.md, pyproject.toml, environment.yml, requirements.txt
|- docs/            course_design.md, infrastructure.md, capstone.md,
|                   resources.md, textbook/ (this book)
|- src/eo_cnn/      installable reference package
|   |- data/        eurosat.py, sentinel2.py, transforms.py
|   |- models/      simple_cnn.py, resnet.py, unet.py
|   |- training/    trainer.py, metrics.py
|   |- visualization/ plots.py
|- notebooks/       chapter_01_.../ ... chapter_12_.../ capstone/
|- data/            (gitignored; auto-created by torchgeo)
```

The `eo_cnn` package is the clean reference implementation of every concept the notebooks teach inline: `data` (EuroSAT/Sentinel-2 loaders, tiling, albumentations pipelines), `models` (SimpleCNN, ResNet classifiers, from-scratch and SMP U-Nets), `training` (classification and segmentation trainers, metrics) and `visualization` (batch grids, prediction triplets, confusion matrices, land-cover maps). The teaching notebooks are deliberately self-contained for Colab; treat `eo_cnn` as the polished version to read after a chapter and to build on in the capstone.

A.4 Hardware recommendations

Chapters	Recommended	Minimum	Colab free
1–4	CPU	CPU	CPU
5	GPU 4 GB+	CPU (slow)	T4
6–8	GPU 6 GB+	GPU 4 GB	T4
9–10	GPU 8 GB+	GPU 6 GB	T4 (batch 4)
11–12	GPU 8 GB+	GPU 6 GB	T4
Capstone	GPU 8 GB+ / A100	GPU 6 GB	Colab Pro A100

The `FAST_MODE` flag (auto-enabled without CUDA) reduces batch size, epochs and dataset fraction so every notebook completes on CPU in under an hour, at reduced accuracy. Dataset disk budget: EuroSAT RGB 90 MB, EuroSAT-MS 500 MB, LoveDA ∼3 GB (all fit Colab's free disk); a BigEarthNet subset should live on Google Drive.

Platform notes. On Windows install GDAL (e.g. via OSGeo4W) before `rasterio`; on Linux/-Mac/Colab `pip install rasterio` works directly. `torchgeo` caches datasets on first use and loads instantly afterwards; all course datasets are freely accessible without registration.

Chapter B

Resources and Further Reading

B.1 Foundational papers

CNN architectures. LeCun et al. (1998, LeNet-5) — the conv-pool-FC template; Krizhevsky et al. (2012, AlexNet) — ReLU, dropout, GPU training; Simonyan & Zisserman (2014, VGG) — depth from stacked 3×3; He et al. (2015, ResNet) — residual connections; Tan & Le (2019, EfficientNet) — compound scaling; Dosovitskiy et al. (2020, ViT) — attention in vision.

Semantic segmentation. Long et al. (2014, FCN) — first fully convolutional segmentation; Ronneberger et al. (2015, U-Net) — encoder–decoder with skips; Chen et al. (2018, DeepLabV3+) — ASPP + decoder; Lin et al. (2017, FPN) — feature pyramids; Zhao et al. (2017, PSPNet) — pyramid pooling.

EO / remote-sensing deep learning. Helber et al. (2019, EuroSAT); Wang et al. (2021, LoveDA); Sumbul et al. (2019, BigEarthNet); Schmitt et al. (2019, SEN12MS); Zhu et al. (2017) and Ma et al. (2019) surveys of DL in remote sensing; Rolf et al. (2021) on distribution shift in satellite ML.

Transfer learning. Yosinski et al. (2014, transferability of features); Neyshabur et al. (2020, what is transferred); Marmanis et al. (2015, ImageNet→EO transfer).

B.2 Courses and documentation

Resource	Focus
fast.ai Practical Deep Learning	practical DL with PyTorch
Stanford CS231n	CNN theory from fundamentals
PyTorch tutorials	framework implementation
torchgeo docs	EO datasets, GeoDatasets
SMP docs	segmentation model library
rasterio user guide	geospatial raster I/O
ESA EO College	remote-sensing background

B.3 Visualisation and GIS tools

In-notebook: `matplotlib` (all plots), `seaborn` (confusion matrices), `plotly`/`folium`/`ipyleaflet` (interactive maps), `earthpy` (band composites), `rasterio.plot.show` (quick GeoTIFF view). External GIS: QGIS (open prediction GeoTIFFs over OSM), Google Earth Pro, ArcGIS Online, GeoServer (serve as WMS/WCS).

B.4 Deployment ideas

A FastAPI `/predict` endpoint that ingests a GeoTIFF and returns a prediction map; a Streamlit dashboard (upload tile → map → download); Google Earth Engine asset export; a Docker image

based on a CUDA PyTorch base; ONNX export for edge deployment on Jetson/OpenVINO.

B.5 Portfolio suggestions

Structure a public repo as `README` (banner + headline metric), `model_card.md` (architecture, training, limitations), executed notebook, 2-page report, and an `assets/` folder (RGB tile, prediction GeoTIFF, confusion matrix). A short GIF of the sliding-window inference progressively covering a scene is a compelling LinkedIn/portfolio post.

B.6 Future advanced topics

Vision transformers for EO (SatMAE, Scale-MAE, Spectral-GPT); self-supervised pretraining on Sentinel-2 (DINO, MAE); change detection (FC-EF, BIT-CD); object detection in aerial imagery (DOTA, oriented R-CNN, YOLO); time-series models (U-TAE, SITS-BERT); EO foundation models (Prithvi, RemoteCLIP, GeoChat); multi-modal SAR+optical and hyperspectral fusion; instance segmentation of building footprints; continual learning across geographies.

B.7 Dataset licenses

Dataset	License	Commercial use
EuroSAT	MIT	yes
LoveDA	CC-BY-4.0	yes (attribution)
BigEarthNet	CC-BY-4.0	yes (attribution)
SpaceNet	CC-BY-SA-4.0	attribution required
Sentinel-2	Copernicus Open Access	yes
OpenStreetMap	ODbL	attribution required
Dynamic World labels	CC-BY-4.0	yes (attribution)

End of textbook. Now go and map the planet.